

**Федеральное государственное бюджетное
образовательное учреждение высшего профессионального
образования «Поволжский государственный университет
телекоммуникаций и информатики»**

Стефанова И.А.

Изучение системы MATLAB+Simulink

Учебно-методическое пособие

Для студентов телекоммуникационных направлений:
«Информационные технологии и системы связи» (11.03.02),
«Фотоника и оптоинформатика» (12.03.03),
«Радиотехника» (11.03.01)

Самара
2015

Оглавление

<i>Введение</i>	3
<i>Рекомендуемая литература</i>	3
<i>Содержание отчета</i>	4
<i>1. Простые вычисления</i>	4
Пример контрольного теста «Основы MATLAB»	14
<i>2. Матричные операторы и функции</i>	16
Пример контрольного теста «Матричные операции»	24
<i>3. Решение линейных уравнений</i>	26
Пример контрольного теста «Линейные уравнения»	36
<i>4. Решение нелинейных уравнений</i>	37
Пример контрольного теста «Нелинейные уравнения»	48
<i>5. Программирование в MATLAB</i>	49
Пример контрольного теста «Программирование»	58
<i>6. Структуры ветвления</i>	61
Пример контрольного теста «Ветвления»	68
<i>7. Циклические структуры</i>	70
Пример контрольного теста «Циклы»	77
<i>8. Моделирование электрических цепей</i>	79
Пример контрольного теста «Электрические цепи»	92

Введение

Данный цикл лабораторных работ включает в себя восемь работ, направленных на освоение математической системы MATLAB и пакета расширения Simulink. Цикл может использоваться на практических занятиях по дисциплинам «Информатика» и «Программирование на языках высокого уровня» при подготовке бакалавров направлений телекоммуникации, радиотехники и оптоинформатики.

Настоящее методическое пособие поможет студентам сориентироваться в учебном материале по изучению математических пакетов дисциплин «Информатики» и «Программирование на языках высокого уровня» и успешно выполнить учебный план указанных дисциплин в целом.

Рекомендуемая литература

Основная

1. Поршнев С. В. MATLAB 7. Основы работы и программирования. Учебник. М.: Бином, 2006, 320 с.
2. Дьяконов В. П. MATLAB 6/5.SP1/7+Simulink 5/6. Основы применения. М.: СОЛОН-Пресс, 2005, 800 с.
3. Дьяконов В. П. MATLAB 6/5.SP1/7+Simulink 5/6. в математике и моделировании. М.: СОЛОН-Пресс, 2008, 576 с.

Дополнительная

4. Кетков Ю. Л. и др. MATLAB 7. Программирование, численные методы. СПб: БХВ-Петербург, 2005, 752 с.
5. Акчурина Э. А. Программирование в системе MATLAB. Методические указания к лабораторным работам, ПГАТИ: Самара, 2004, 50 с.
6. Конспект лекций по дисциплинам «Информатика», «Программирование на языках высокого уровня».

Содержание отчета

1. Название работы, цель работы, задание в соответствии с вариантом.
2. Блок-схем алгоритма в соответствии с номером варианта к работам 5, 6, 7.
3. Программа вычислений.
4. Результаты выполнения программы на ПК.
5. Выводы.

1. Простые вычисления

1.1. Подготовка к работе

По указанной литературе изучить:

- интерфейс пользователя, окна системы MATLAB;
- правила ввода текста, данных и переменных;
- операторы и функции;
- задание функций пользователя;
- правила вывода результатов;
- построение графиков и графические средства для работы с ними;
- построение графиков нескольких функций в одном окне и в разных окнах.

1.2. Задание и порядок выполнения работы

1. Запустить систему MATLAB. Изучить структуру интерфейса.
2. В главном окне последовательно открыть все пункты меню и усвоить место положения основных команд.
3. Последовательно установить указатель мыши на каждую из кнопок панелей инструментов и усвоить выполняемые ими команды.
4. В командном окне ввести текстовый комментарий "Простые вычисления".
5. Задача 1. В режиме прямых вычислений в командной строке поочередно вывести значения: π , realmax , realmin , eps , $\sqrt{12}$, $\sin(\pi/6)$, calendar .

6. Задача 2. Построить графики заданных функций по исходным данным, приведенным в табл. 1.1 и в табл. 1.2.

Для этого:

1. задать исходные данные x_0 , x_n , h ;
2. задать ранжированную переменную X , изменяющуюся в пределах от x_0 до x_n с шагом h ;
3. ввести функции пользователя Y и Z ;
4. вычислить и вывести значения функций Y и Z для аргумента X в заданном интервале;
5. вывести графики функций Y и Z в декартовых координатах:
 - в разных окнах (Y – figure1, Z – figure2);
 - вместе в одном окне (Y, Z – figure3);
 - в двух подокнах (Y – figure4(1), Z – figure4(2)) в виде линий,
 - в двух подокнах (Y – figure4(3), Z – figure4(4)) в виде столбиков (гистограмм).

Для графиков функций Y и Z использовать разный тип линий. Для первого графика $Y(X)$ цвет линии, тип точек и тип линии взять по варианту N из таблицы 1.2, для второго графика $Z(X)$ – взять характеристики линии из той же таблицы, но по варианту $13 - N$.

6. Добавить в каждый график заголовок и сетку. Для графиков, построенных совместно в одном окне добавить легенду.
7. Проанализировать полученные результаты на соответствие поставленных задач.
8. Программу и графики функций $Y(X)$ и $Z(X)$ скопировать в отчет, созданный в текстовом редакторе MS Word. Сохранить его в своей папке.
9. Сделать выводы о вводе текстовых комментариев, видах переменных, правилах построения математических выражений и графиков, графических возможностях системы.

Таблица 1.1. Варианты функций.

N	Функция 1	Функция 2	x ₀	x _n	h	a	b
1	$y=a \cdot \cos(x)$	$z = \frac{x^2(x+2)}{b} - b \sin^2(x+2)$	-2,5	2	0,3	2	5
2	$y=\sin(x^2)$	$z = \sqrt{\frac{(x+b)^2}{a}} + \cos(x+b)$	-4	4	0,2	2	4
3	$y= \operatorname{tg}(x) /x^2$	$z = \sqrt{a \sin x + b \sin x }$	-3,0	3,0	0,4	3	5
4	$y=x^2-a$	$z = \frac{a^{2x} + \cos(a+b)x}{\sin(a+b)x}$	-3,5	-0,3	0,3	5	5
5	$y=x^3/a$	$z = e^{-bx} \frac{x + \sqrt{(x+a)^2}}{x - \ln x+a }$	-2,4	-0,6	0,2	3	1,5
6	$y=x^2 - 5$	$z = \sin^3(x^2+a) - \sqrt{\frac{x^2+a}{b}}$	-3	3	0,3	2	5
7	$y=b \sin^2(x)$	$z = (a-x) \frac{b(a-x)}{1+(a-x)^2}$	-3	2	0,5	-1	10
8	$y=a \cdot \cos(x)$	$z = \ln(a+x^2) + \sin \frac{(a+x^2)}{b}$	-3,5	3,5	0,2	5	2
9	$y=\sin(bx/a)$	$z = e^{-bx} \sin(ax+b) - \sqrt{ ax+b }$	-2,4	1,5	0,4	-0,5	2
10	$y=a \cdot \sin(ax)$	$z = x^{b/x} - \sqrt{ b/x }$	-0,5	2	0,2	2	0.15
11	$y=b \cdot \sin^2(x/3)$	$z = \sqrt{x^2+b} - \frac{\cos(x^2+b)}{a}$	-5	5	0,2	-0,5	15
12	$y=2 \cos(x^2)$	$z = e^{-ax} \sqrt{ x +1} + e^{-bx} \sqrt{ x +1}$	-5	5	0,3	0,5	0,2

Таблица 1.2. Варианты атрибутов графиков.

N	Тип линии	Цвет линии	Тип маркера	N	Тип линии	Цвет линии	Тип маркера
1	-	y	.	7	-.	r	d
2	:	m	o	8	--	k	v
3	-.	c	x	9	-	y	<
4	--	r	+	10	:	m	>
5	-	g	*	11	-.	c	p
6	:	b	s	12	--	r	h

1.3. Методические указания по выполнению задания

В MATLAB используют два режима работы:

- прямых вычислений – в командном окне. В этом случае каждое действие сразу же исполняется;
- в редакторе программ. В этом случае программа вводится в окно редактора, сохраняется, а исполняется по команде встроенного отладчика.

При работе в режиме прямых вычислений выражения могут вводиться:

- в прямой форме, тогда после завершения ввода ответ будет выведен под встроенным системным именем **ans**. Переменная с этим именем всегда хранит результат последнего вычисления.
- в форме оператора присвоения, когда переменной с выбранным именем присваивается значение выражения. Ответ в этом случае выводиться под именем этой переменной.

Символ присвоения в системе – знак равенства (=).

Любое уже определенное значение можно вызвать из рабочей области по имени переменной.

Если вычисляется значение переменной с выбранным именем по заданному выражению, результат выводится под именем этой переменной в следующей строке.

Вывод результата можно заблокировать, если в конце строки ввода ввести знак точка с запятой (;).

В MATLAB переменным можно задать определенные значения, типы которых заранее не декларируются. Тип результата определяется автоматически по виду выражения.

В идентификаторах имеет значение высота буквы. Для имен простых переменных и функций используют строчные буквы, а для структурированных переменных (векторов и матриц) заглавные.

Если данные не умещаются в строке, строку можно отобразить в нескольких строках, используя разделитель в виде многоточия (не менее трех точек).

Текстовые пояснения в программу вводятся, как комментарий. Он начинается с символа %, который располагается обычно в первой позиции строки.

В MATLAB имеются ряд системных переменных, которые могут быть переопределены:

- i или j – мнимая единица,
- pi – число $\pi = 3,1415926\dots$,
- eps – погрешность операций над числами с плавающей точкой,
- realmin – наименьшее число с плавающей точкой,
- realmax – наибольшее число с плавающей точкой,
- inf – значение машинной бесконечности.

Значение переменной, результат которой присваивается, хранится в рабочей области.

Ранжированная переменная применяется для построения графиков, создания циклов с известным числом повторений и имеет ряд фиксированных значений, меняющихся от начального значения x_0 до конечного x_n с определенным шагом h .

Формат задания ранжированной переменной:

$$X = x_0 : h : x_n ;$$

Функция имеет имя и список аргументов, заключенных в круглые скобки, например $\sin(X)$. В данном случае аргументом является ранжированная переменная – вектор X .

В системе MATLAB имеется большое количество встроенных математических функций, общий перечень которых можно получить в командном окне командами `help elfun`, некоторая часть из которых приведена в таблице 1.3.

Таблица 1.3. Элементарные функции

Функция	Описание	
abs(X)	Абсолютное значение X	X
exp(X)	Экспонента от X.	e^X
expm1(X)	Экспонента от X минус 1.	$e^X - 1$
cos(X)	Косинус	cosX
sin(X)	Синус	sinX
tan(X)	Тангенс	tgX
cot(X)	Котангенс	ctgX
log(X)	Натуральный логарифм от X	lnX
log10(X)	Логарифм от X по основанию 10	lgX
sqrt(X)	Квадратный корень из X..	$\sqrt{X}$
X.^n	Возведение X в степень n	X^n

Для правильного вызова встроенных функций используется справка MATLAB Help и пункт меню Function Browser. В окне категории выделить нужный раздел, а в нем список функций. В списке найти нужную функцию и щелкнуть по ней. В дополнительном окне отобразится описание выбранной функции с форматом обращения и примерами использования, а при двойном щелчке по функции запись ее перемещается в командную строку.

На основе встроенных элементарных функций строятся функции пользователя.

Формат задания функции пользователя:

Имя = Выражение;

Для вычисления функции и вывода таблицы ее значений необходимо ввести ранжированную переменную, задать функцию пользователя, и вывести значения клавишей <Entr>.

При работе с массивами определены операторы поэлементного выполнения. В последнем случае, перед символом операции вводится точка (.). Например, функция $y = e^{-x} \cdot \sin^2(|x^3 + 5|)$ в MATLAB будет записана как $Y = \expm1(X) .* \sin(\text{abs}(X.^3 + 5)).^2$; где вектор X в виде ранжированной переменной определен ранее.

Для построения графика необходимо ввести ранжированную переменную с заданным шагом, задать функцию пользователя, вызвать окно для построения графика функции `figure()` и с помощью команды `plot` построить график.

Процедура `plot(x,y,s)` рисует график прямыми линиями между вычисленными точками. Здесь `s` - строковая константа, задающая параметры линии, ее можно пропускать. В MATLAB определены следующие значения строковой константы `s`:

Таблица 1.4. Значения строковой константы `s`:

Тип линии		Цвет линии		Тип маркера	
-	сплошная	y	желтый	.	точка
:	двойной пунктир	m	фиолетовый	o	кружок
-.	штрих пунктир	c	голубой	x	крест
--	штрих	r	красный	+	плюс
		g	зеленый	*	звездочка
		b	синий	s	квадрат
		w	белый	d	ромб
		k	черный	v	треугольник вниз
				^	треугольник вверх
				<	треугольник влево
				>	треугольник вправо
				p	пятиугольник
				h	шестиугольник

Если на одном графике нужно отобразить несколько функций, например, $Y_1=f(X)$ и $Y_2=f(X)$, то они вначале вычисляются, а затем выводятся командой `plot(X,Y1,'s1',X,Y2,'s2')`, в которой в качестве параметров для каждой функции следуют группы <аргумент, функция, строка типа линии>.

Например, `plot(X,Y1,'mp',X,Y2,'--bh')`.

Эта строка позволяет нарисовать два графика $Y_1(X)$ и $Y_2(X)$ с разными атрибутами линий в одном окне (см. табл. 1.4).

Для создания в графическом окне нескольких подокон для вывода графиков используется процедура `subplot(m,n,p)`, где `m` - число подокон в окне по горизонтали, `n` - по вертикали, `p` - номер используемого подокна (нумерация с 1).

Для формирования графика в столбиковой форме нужно использовать команду `bar(X,Y)`. При выводе такого графика в подокно строка программы имеет вид `subplot(m,n,p), bar(X,Y)`.

Пример выполнения некоторых заданий в режиме прямых вычислений:

```
>> % Простые вычисления
```

```
>> pi
```

```
ans =
```

```
3.1416
```

```
>> sqrt(21)
```

```
ans =
```

```
4.5826
```

```
>> calendar
```

```
Feb 2014
```

S	M	Tu	W	Th	F	S
0	0	0	0	0	0	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	0
0	0	0	0	0	0	0

```
>>% Ввод исходных данных для построения графика
```

```
>> xo=-6;
```

```
>> xn=6;
```

```
>> h=0.3; % Шаг
```

```
>> X=xo:h:xn; % Задание аргумента
```

```
>> a=2;
```

```
>> % Расчет функций
```

```
>> Y=abs(cos(X));
```

```
>> Z=-abs(sin(a*X));
```

```
>> % Вывод графиков с разными типами линии в окно1
```

```
>> figure(1);
```

```
>> plot(X,Y,'mo');
```

```

>> title('Y,X'); % Добавление заголовка
>> % Вывод графиков с разными типами линии в окно2
>> figure(2);
>> plot(X,Z,'--bd');
>> title('Z,X'); % Добавление заголовка
>> figure(3);
>> plot(X,Y,':mo',X,Z,'--bd');
>> grid on; % Добавление координатной сетки
>> title('Y,Z'); % Добавление заголовка
>> legend('Function Y', 'Function Z'); % Добавление легенды
>> figure(4);
>> % Вывод графика Y в виде линий подокно1
>> subplot(2,2,1), plot(X,Y,':mo');
>> title('X,Y');
>> % Вывод графика Z в виде линий подокно3
>> subplot(2,2,3), plot(X,Z,'--bd');
>> title('X,Z');
>> % Вывод графика Y в виде столбиков подокно2
>> subplot(2,2,2), bar(X,Y);
>> title('X,Y Hist');
>> % Вывод графика Z в виде столбиков подокно4
>> subplot(2,2,4), bar(X,Z);
>> title('X,Z Hist');

```

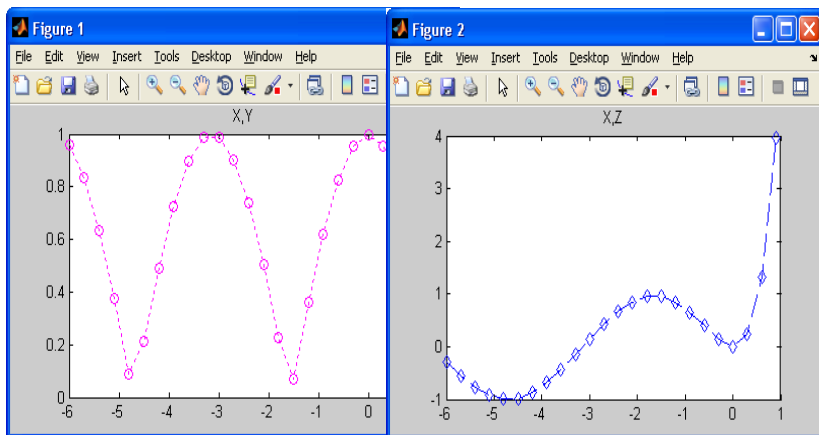


Рис.1. Вывод графиков с разными типами линии в окна 1 и 2.

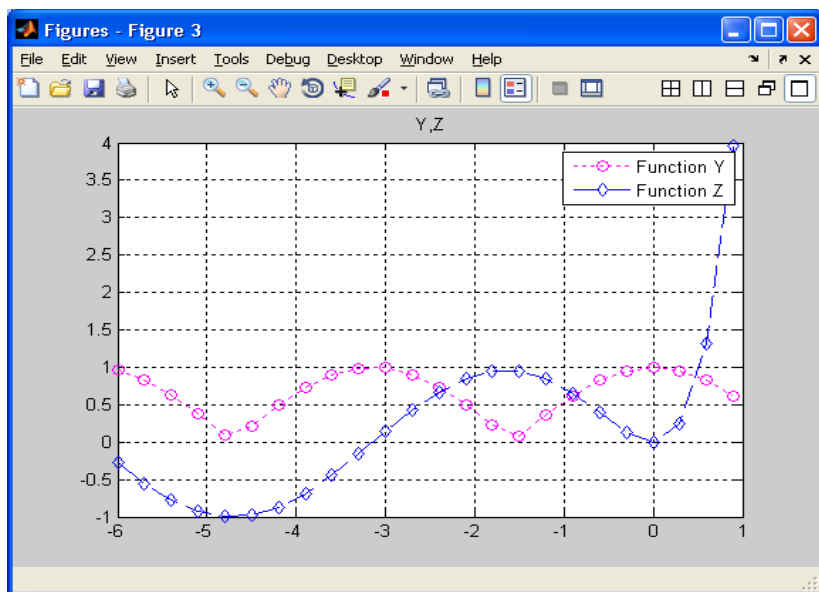


Рис.2. Вывод графиков с разными типами линии в окно3.

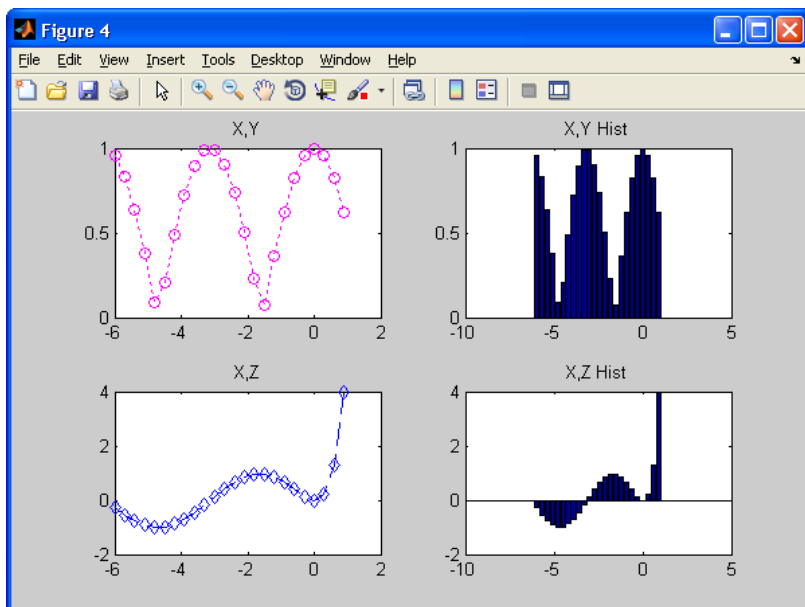


Рис.3. Вывод графиков в разные подокна окна 4.

Пример контрольного теста «Основы MATLAB»

Матричным оператором в системе MATLAB не является:

+ * *. .* -

Укажите оператор формирования упорядоченных последовательностей в диапазоне от A до B с шагом H в системе MATLAB:

X=A, A+H..B

A, H.. B

A: H: B

A: A+H: B

A..A+H..B

Укажите 3 оператора сравнения в MATLAB:

== <> >= ~= =<

Укажите все логические операторы в MATLAB:

* & | + ~

Укажите оператор формирования упорядоченных числовых последовательностей в MATLAB:

.. : ,... ... =

Оператор переноса строки длинного математического выражения в MATLAB:

.. : %

Выражение $t=a^{2x}+x^k$ на языке MATLAB:

$t=\exp(2*x*\ln(a))+\exp(x*\ln(k))$

$t=2*x^a+x^k$

$t=a^{2*x}+x^k$

$t=\exp(2*x*\ln(a)+(x*\ln(k)))$

$t=a^{(2*x)}+x^k$

Укажите 4 системные переменные, используемые MATLAB:

i ans pi j exp

В алфавит языка MATLAB не входят:

буквы латинского алфавита
буквы русского алфавита
специальные символы
зарезервированные слова
стандартные идентификаторы

Укажите все правильные идентификаторы:

k1 1x pi д t

Установите порядок выполнения операций в MATLAB по убыванию их приоритета

вычисления в ()

транспонирование и возведение в степень

* /

+ -

= <> < > <= >=

& |

Укажите 3 стандартные функции MATLAB:

ln(x) log(x) exp(x) sqr(x) sqrt(x)

Выражение не содержит:

круглых скобок

данных, над которыми выполняются действия

знаков операции

фигурных скобок

функций

Введите символ, запрещающий вывод результата в MATLAB:

Введите символ обозначения комментария в системе MATLAB:

Введите ранжированную переменную X в диапазоне от a до b с шагом h в системе MATLAB.

2. Матричные операторы и функции

2.1. Подготовка к работе

По указанной литературе изучить:

- создание матриц и векторов;
- матричные операторы и функции;
- правила вывода результатов;
- специальные символы и операции с матрицами;

2.2. Задание и порядок выполнения работы

1. Запустить систему MATLAB и ввести текстовый комментарий "Работа с матрицами и векторами".
2. Задача 1. В режиме прямых вычислений выполнить над векторами V1 и V2 , приведенными в табл.2.1, следующие действия:
 - ввод векторов V1, V2;
 - сложение $V3=V1+V2$ и вычитание векторов $V4=V1-V2$;
 - по элементное умножение $V5=V1.*V2$ и по элементное деление заданных векторов $V6= V1./V2$;
 - по элементное возведение элементов вектора V1 в степень $k = 3$;
 - транспонирование вектора V1;
 - умножение вектора V2 на скаляр s (значение которого приведено в табл. 2.1);
 - деление полученного результата на скаляр s;
 - вывод 3-го элемента от результата предыдущей операции;
 - удаление 1-го элемента вектора V2;
 - вывод результата последнего действия в виде столбца;
3. Задача 2. В режиме прямых вычислений выполнить над матрицами M1 и M2, приведенными в табл.2.1, следующие действия:
 - ввод матриц M1 и M2;
 - сложение $M3=M1+M2$ и вычитание матриц $M4=M1-M2$;
 - матричное умножение и по элементное умножение заданных матриц;

- матричное деление и по элементное деление заданных матриц;
- по элементное возведение элементов матрицы M1 в степень $k = 2$;
- транспонирование матрицы M1;
- умножение матрицы M2 на скаляр $s = 5$ (значение которого приведено в табл. 2.1),
- деление полученного результата на скаляр s ;
- вывод 3-й строки матрицы M1
- вывод 2-го столбца матрицы M1;
- вывод последнего элемента матрицы M2;
- удаление 1-й строки матрицы M2;
- и удаление 3-го столбца матрицы M2;
- вывод элементов матрицы M1, лежащих на побочной диагонали с использованием имени массива и индексации элементов.

Таблица 2.1. Варианты заданий.

N	V1	V2	s	M1	M2
1	[1 -6 5 4]	[10 -57 52 16]	5	[5 2 -3; 4 -7 2; -5 3 9]	[-5 3 8; 5 -6 6; 7 6 -9]
2	[-2 7 4 5]	[11 35 -64 80]	3	[1- 5 4; -2 3 7; 6 9 -4]	[3 9 -6; -6 7 3; 1 -8 5]
3	[2 -5 9 6]	[15 14 74 -35]	7	[9 -3 4; -5 7 2; -1 8 5]	[2 -2 7; 4 8 -3; -5 6 1]
4	[4 3 -6 8]	[-14 76 98 67]	6	[-8 5 1; 4 -2 6; 7 3 -9]	[1 9 -5; 6 -3 6; 8 4 -2]
5	[-5 3 6 7]	[13 -34 64 98]	2	[6 -2 9; 7 5 -6; 3 5 -1]	[-8 5 2; -6 9 5; -1 4 3]
6	[7 -4 6 8]	[16 45-38 54]	4	[-4 2 8; 5 -3 7; -5 6 9]	[4 6 -8; 8 5 -6; 2 3 -7]
7	[9 5 -3 6]	[17 48 96 -46]	8	[7 2 -1; -5 3 8; 5 -6 6]	[5 2 -3; 4 -7 2; -5 3 9]
8	[-8 6 4 5]	[-12 55-47 96]	5	[2 5 -8; 3 9 -6; -6 7 3]	[-2 3 7; 6 9 -4; 2 -9 4]

9	[7 -4 8 6]	[19-35 90 -11]	6	[-5 6 9; 2 -2 7; 4 8 -3]	[-5 7 2; -1 8 5; -3 9 1]
10	[-1 5 9 4]	[-20 46 38-65]	3	[2 -4 7; 1 9 -5; 6 -3 6]	[4 -2 6; 7 3 - 9; 5 1 8]
11	[3 9 6 -5]	[21-64 -51 45]	4	[1 3 -7; -8 5 2; -6 9 5]	[7 5 -6; 3 5 -1; 4 8 -9]
12	[2 6 3 -6]	[-22 75 49 52]	7	[-2 3 7; 4 6 -8; 8 5 -6]	[5 -3 7; -5 6 9; 1 8 -2]

4. Задача 3. Используя векторные и матричные функции (см. табл. 2.4) вычислить:

- длину L вектора $V1$;
- сумму SV элементов вектора $V1$;
- суммы (S) и произведения (P) элементов матрицы $M1$ по столбцам ($S1$ и $P1$) и по строкам ($S2$ и $P2$);
- средние арифметические значения элементов вектора $V1$ и матрицы $M1$;
- максимальные и минимальные значения вектора $V2$ и матрицы $M2$;
- элементы матрицы $M1$ на главной диагонали;
- определитель (детерминант) матрицы $M1$;
- след матрицы $M1$ (сумма элементов, лежащих на главной диагонали квадратной матрицы);
- отсортировать вектор $V1$ в порядке возрастания его элементов, а вектор $V2$ в порядке убывания элементов;
- отсортировать матрицу $M1$ в порядке возрастания ее элементов сначала по строкам, а затем и по столбцам.

5. Проанализировать полученные результаты на соответствие поставленных задач.

6. Программу расчета векторных и матричных операторов и функций скопировать в отчет, созданный в текстовом редакторе MS Word. Сохранить его в своей папке.

7. Сделать выводы о двойственности математических операторов и функций, значении специальных символов, возможностях матричных и векторных функций для вычисления в массивах данных.

2.3. Методические указания по выполнению задания

В MATLAB все данные рассматриваются, как матрицы, чем обеспечивается высокая скорость математических действий.

Матрицы могут задаваться с помощью оператора присвоения, когда переменной с выбранным именем присваивается значение выражения. Ответ в этом случае выводится под именем этой переменной. В случае вывода результата операции без использования имени новой переменной, он выводится под именем системной переменной **ans**. Переменная с этим именем всегда хранит результат последнего вычисления.

Элементы одномерного массива могут вводиться:

- в квадратных скобках, разделяясь пробелами. Например, вектор $V=[1\ 2\ 4\ 8]$;
- поочередно, с использованием имени массива и индекса элемента. Например, $V(1)=1$; $V(2)=2$; $V(3)=4$; $V(4)=8$.

Элементы двумерного массива могут вводиться:

- в квадратных скобках, внутри которых размещаются векторы строк, разделенные знаком точка с запятой (;). Например, матрица $M=[1\ -2\ -3; 4\ 5\ 6; 7\ -8\ 9]$;
- последовательно, с использованием имени массива и индексов строки и столбца на пересечение, которого, лежит элемент. Например, $M(1,1)=1$; $M(1,2)=-2$; $M(1,3)=-3$; $M(2,1)=4$; $M(2,2)=5$; .. $M(3,3)=9$.

Для работы с массивами используются специальные символы, приведенные в табл. 2.2:

Таблица 2.2. Специальные символы и операторы

Оператор	Описание
$V(n)$	вывод n – го элемента вектора V
$V(:)$	вывод вектора в столбец
V'	транспонирование вектора / матрицы
$A(m,n)$	вывод m,n – го элемента матрицы A
$A(m,:)$	вывод строки m из матрицы A .
$A(:, n)$	вывод столбца n из матрицы A
$A(m,:) = []$	удаление строки m из матрицы A
$A(:,n) = []$	удаление столбца n из матрицы A

Большинство операторов относятся к матричным операторам, например $+$, $-$, $*$, $/$.

При работе с массивами определены операторы поэлементного выполнения. В последнем случае, перед символом операции вводится точка ($.$). Например, $X.^3$ (возведение элементов массива X в третью степень). Символ присвоения – знак равенства ($=$).. Например, $Y=X.^3$.

В таблице 2.3 приведены арифметические матричные операторы MATLAB.

Таблица 2.3. Арифметические матричные операторы

Оператор	Описание действий
$A + B$	Сложение. A , B должны быть одинакового размера
$A - B$	Вычитание. A , B должны быть одинакового размера
$A * B$	Матричное умножение.
$A .* B$	Поэлементное умножение массивов.
A / B	Деление матриц слева на право. A делится на B .
$A ./ B$	Поэлементное деление массивов слева на право (A/B)
$A.^B$	Поэлементное возведение массива в степень.
$A.'$	Транспонирование массива.
$A:H:B$	Список от A до B с шагом H .
$A : B$	Список от A до B с шагом 1.

Функция имеет список аргументов, заключенных в круглые скобки, например $\min(X)$, где вектор X – аргумент. В системе MATLAB имеется большое количество встроенных математических функций, перечень которых можно получить в командном окне командой `help matfun`. В таблице 2.4 приведены встроенные векторные и матричные функции MATLAB.

Таблица 2.4. Векторные и матричные функции

Функция	Описание
$\text{length}(V)$	возвращает длину вектора V
$\text{sum}(V)$	возвращает сумму элементов вектора V

sum(M,1)	возвращает сумму элементов матрицы M по столбцам
sum(M,2)	возвращает сумму элементов матрицы M по строкам
prod(M,1)	возвращает произведение элементов матрицы M по столбцам
prod(M,2)	возвращает произведение элементов матрицы M по строкам
max(V)	возвращает максимальное значение элементов вектора V
min(V)	возвращает минимальное значение элементов вектора V
mean(V)	возвращает среднее арифметическое элементов вектора V
sort(V) -sort(-V)	располагает элементы вектора V по возрастанию и по убыванию соответственно
diag(M)	возвращает элементы, лежащие на главной диагонали матрицы M
det(M)	определитель (детерминант) матрицы M
trace(M)	след матрицы M (сумма диагональных элементов)
mean(M,1)	возвращает среднее арифметическое значение элементов матрицы M по столбцам
mean(M,2)	возвращает среднее арифметическое значение элементов матрицы M по строкам

* Примечание. В табл. 2.4. функции, содержащие в качестве второго аргумента единицу (1) возвращают действие по столбцам, а двойку (2) – по строкам. Это правило можно применить и для вывода максимального/минимального/среднего значения элементов в столбце max(M,1) и строке матриц min(M,2), а так же произвести их сортировку по столбцам sort(M,1) или строкам sort(M,2) соответственно.

Примеры работы с векторами и матрицами:

```
>> % Работа с матрицами и векторами
>> V1=[12 -57 52 6];
```

```

>> V2=[14 17 98 62];
>> V3=V1+V2 % Сложение двух векторов
V3 =
    26   -40   150    68
>> V4=V1-V2 % Вычитание двух векторов
V4 =
    -2   -74   -46   -56
>> s=5;
>> V5=s*V1 % Умножение вектора на скаляр
V5 =
    60  -285   260    30
>> V6=V5./s % Деление вектора на скаляр
V6 =
    12   -57    52     6
>> max(V1) % Вывод максимального элемента вектора V1
ans =
    52
>> mean(V1) % Вывод среднеарифметического значения
элементов вектора V1
ans =
    3.2500
>> V1.^2
ans =
    144    3249    2704     36
>> V1(:) % Вывод элементов вектора V1
ans =
    12
   -57
    52
     6
>> sort(V1) % Сортировка элементов вектора V1
ans =
   -57     6    12    52
>> M1=[4 -7 2; -5 3 9] % Ввод и вывод матрицы M1
>> M1 =
     4    -7     2
    -5     3     9
>> M1(1,2) % Вывод элемента на пересечении 1 строки и 2 столбца

```

```

ans =
    -7
>> mean(M1,1)    % Среднее значение по столбцам матрицы M1
ans =
    -0.5000    -2.0000    5.5000
>> sum(M1,2)     % Сумма элементов по строкам матрицы M1
ans =
    -1
     7
>> M2=[4 -7 2; -5 3 9; 1 0 8] % Ввод и вывод матрицы M2
M2 =
     4    -7     2
    -5     3     9
     1     0     8
>> diag(M2)      % Элементы, лежащие на главной диагонали M2
ans =
     4
     3
     8
>> P=prod(M2,2) % Произведение элементов матрицы M2 по
строкам
>> P
P =
    -56
   -135
     0
>> det(M2)       % Определитель матрицы M2
ans =
   -253
>> trace(M2)     % След матрицы M2
ans =
    15
>>
>> sort(M2,1)    % Сортировка матрицы M2 по столбцам
ans =
    -5    -7     2
     1     0     8
     4     3     9

```

Пример контрольного теста «Матричные операции»

Укажите правильное задание матрицы D размером 3x3 в MATLAB:

D=[2 3 4; -5 6 4; 0 -7 1]

D=(2 3 4; -5 6 4; 0 -7 1)

D=[2: 3: 4; -5: 6: 4; 0: -7: 1]

D=[2 3 4, -5 6 4, 0 -7 1]

D=[2, 3, 4, -5, 6, 4, 0, -7, 1]

Установите соответствие операторов:

A (: , 1)

A (1 , :)

A (: , 1)=[]

A (1 , :)=[]

вывод 1-го столбца матрицы A

вывод 1-й строки матрицы A

удаление 1-го столбца матрицы A

удаление 1-й строки матрицы A

Вывод на экран вектор V в виде столбца:

V=

V(:)

V(..)

V(')

V.^

Удаление 1-й строки матрицы A

A (1 , :)=[]

A (: , 1)=[]

A (1 , :)

A (: , 1)

A (1 , 1)

Вывод на экран 2-го столбца матрицы B

B (2 , :)=[]

B (: , 2)=[]

B (2 , :)

$B(:, 2)$

$B(2, 2)$

Вывод на экран первого элемента вектора U

$U(1)$ $U[1]$ U_1 $U(1:)$ $U1$

Вывод на экран 3-й строки матрицы B

$B(3, :) = []$

$B(:, 3) = []$

$B(3, :)$

$B(:, 3)$

$B(3, 3)$

Вывод на экран последнего элемента матрицы D размером 4x4:

$D[4,4]$

$D(4,4)$

$D\{4,4\}$

$D[:4,:4]$

$D_{4,4}$

Поэлементное возведение массива G во вторую степень

$G.^2$ G'^2 $G^{\wedge}2$ $G.'^2$ $G^{\wedge}(2)$

Введите функцию, возвращающую наибольший элемент вектора A

Введите функцию, возвращающую наименьший элемент вектора B

Введите функцию, возвращающую среднее арифметическое значение элементов вектора C

Введите функцию, возвращающую суммарное значение элементов вектора D

3. Решение линейных уравнений

3.1. Подготовка к работе

По указанной литературе изучить:

- способы решения линейных уравнений;
- объекты подбиблиотек Math Operations, Sinks, Sources, Signal Processing Blockset /Math Function;

3.2. Задание и порядок выполнения работы

1. Запустить систему MATLAB и ввести текстовой комментарий "Решение системы линейных уравнений" (СЛУ).
2. Задача 1. Решить линейное уравнение (ЛУ) типа $a \cdot x + b = 0$ с последующим его моделированием. Коэффициенты уравнения заданы по варианту в таблице 3.1. Для этого:
 - решить ЛУ в режиме прямых вычислений;
 - создать модель решения ЛУ с использованием блоков математических операций пакета Simulink;
 - сохранить модель и сравнить результаты моделирования ЛУ с вычисленными значениями;
 - скопировать в отчет модель решения ЛУ и результаты проверки решения.

Таблица 3.1. Варианты заданий

N	a	b	N	a	b	N	a	b	N	a	b
1	-1,5	15	5	0,09	7,2	9	0,5	9,48	13	-1,2	13,8
2	0,31	0,95	6	3,8	9,5	10	-0,07	0,58	14	0,51	1,91
3	0,48	-12	7	2,5	-13,3	11	0,4	11,4	15	0,18	-11,8
4	-0,07	11,2	8	-0,6	12	12	2,61	-5,6	16	-0,12	8,24

3. Задача 2. Решить систему линейных уравнений (СЛУ) с последующим ее моделированием:

$$\begin{cases} a_{11} \cdot x_1 + a_{12} \cdot x_2 + a_{13} \cdot x_3 = b_1 \\ a_{21} \cdot x_1 + a_{22} \cdot x_2 + a_{23} \cdot x_3 = b_2 \\ a_{31} \cdot x_1 + a_{32} \cdot x_2 + a_{33} \cdot x_3 = b_3 \end{cases}$$

Коэффициенты уравнений заданы по варианту в таблице 3.2.

Для этого:

- задать матрицу A – из коэффициентов $a_{11} a_{12} \dots a_{33}$,
- задать вектор B – свободных членов b_1, b_2, b_3 ,
- вычислить в командной строке корни СЛУ методом наименьших квадратов (с помощью встроенной функций $lsqr()$);
- создать модель решения СЛУ с использованием блоков матричных операций пакета *Signal Processing Blockset*,
- сохранить модель и сравнить результаты моделирования СЛУ с вычисленными значениями,
- скопировать в отчет модель решения СЛУ и результаты проверки решения матричным способом.

Таблица 3.2. Варианты заданий

N	a_{11}	a_{12}	a_{13}	b_1	a_{21}	a_{22}	a_{23}	b_2	a_{31}	a_{32}	a_{33}	b_3
1	2	0,1	-0,05	10	0,06	-2,5	0,3	12	0,02	-0,3	4	15
2	3	0,2	-0,15	11	-0,02	1,3	-0,6	9,5	-0,04	0,08	6	20
3	-1,5	0,15	-0,22	15	0,31	-5,7	0,75	12	0,08	-0,06	3,5	16
4	4,8	-0,6	0,4	8	-0,05	6,3	-0,4	8,5	-0,07	0,3	5	12
5	-7,2	0,3	0,01	12	-0,16	4,5	0,8	10	0,3	-0,05	6,5	18
6	4,5	0,08	-0,15	9	0,4	-7,2	0,3	11,4	-0,6	0,65	3	14
7	6,3	-0,4	0,21	14	-0,01	4,8	-0,6	9	0,2	0,2	2	19
8	-5,7	0,65	0,35	8,5	0,05	-1,5	0,2	10,5	0,15	-0,07	8	10
9	1,4	-0,7	0,02	11,3	0,15	3	-0,2	15	0,1	0,15	7,5	12
10	-2,5	0,32	0,06	10,5	0,07	2	-0,2	14	0,5	0,1	4,5	13
11	2,5	0,23	-0,2	9,5	0,09	2,4	0,5	8	-0,05	0,09	9,5	11
12	3,8	0,3	-0,3	13	0,08	-3,5	0,7	6	0,2	-0,5	7	17
13	3	0,2	-0,15	11	-0,02	1,3	-0,6	9,5	-0,04	0,08	6	20
14	4,8	-0,6	0,4	8	-0,05	6,3	-0,4	8,5	-0,07	0,3	5	12
15	4,5	0,08	-0,15	9	0,4	-7,2	0,3	11,4	-0,6	0,65	3	14
16	1,4	-0,7	0,02	11,3	0,15	3	-0,2	15	0,1	0,15	7,5	12

4. Сделать выводы относительно величины погрешности вычислений линейных уравнений разными методами.

4.3. Методические указания по выполнению задания 1

1. Решение линейного уравнения (ЛУ) вида $a \cdot x + b = 0$ можно в командной строке MATLAB, например:

```
>> a=3;
>> b=-6.6;
>> x=-b/a
x =
    2.2000
```

- В *Simulink* создайте модель устройства (рис. 3.1), выполняющего решение линейного уравнения. В таблице 3.3 указаны названия блоков, папки в которых находятся нужные блоки для моделирования и параметры блоков, которые нужно изменить:

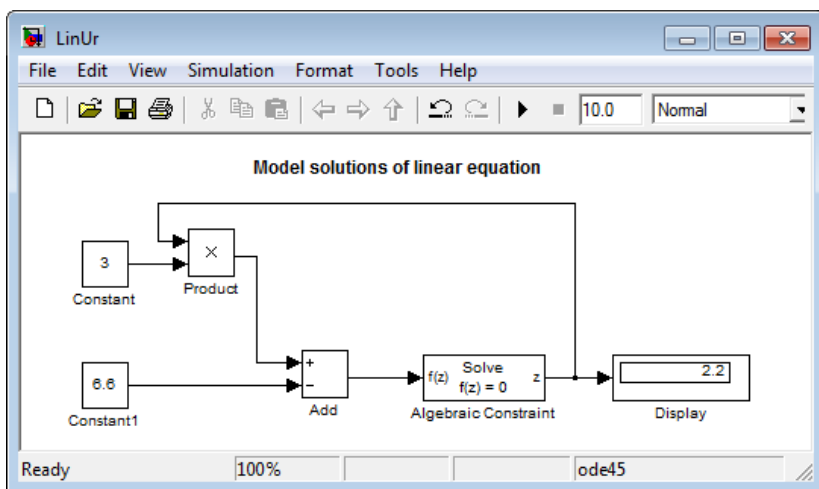


Рис. 3.1. Модель решения линейного уравнения $3 \cdot x - 6,6 = 0$

Таблица 3.3. Значения параметров блоков моделирования ЛУ

Название блоков	Размещение в браузере	Параметры блоков
<i>Constant, Constant1</i>	<i>Simulink / Sources</i>	<i>Constant1=3, //устанавливаются Constant2=6.6 // по варианту</i>
Блоки <i>Add, Product</i>	<i>Simulink/ Math Operations</i>	<i>List of signs: +- // Add Number of inputs: 2 //Product</i>
Блок <i>Algebraic Constraint</i>	<i>Simulink/Math Operations</i>	<i>Initial guess: 2 // значения выявляются по приближенному решению</i>
<i>Display</i>	<i>Simulink/Sinks</i>	По умолчанию

Блок алгебраического ограничения *Algebraic Constraint* служит для решения уравнений или систем уравнений, накладывающих ограничения на значения переменных. Он находит такое значение выходного сигнала (x), при котором значение входного сигнала $F(x)$ становится равным нулю ($F(x)=0$). При этом входной сигнал должен быть связан с выходным сигналом по цепи обратной связи (см. рис. 3.1). Используя свойства блока, целесообразно задать начальное приближение корней уравнения для повышения эффективности его решения. Для уравнения вида $3x - 6,6 = 0$ начальное приближение корня равно 2.

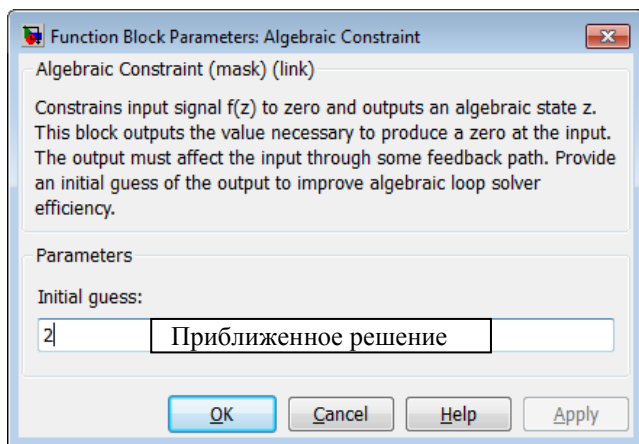


Рис. 3.2. Окно установки параметров блока *Algebraic Constraint*

3. Сохраните модель и запустите ее исполнение. Результаты вычислений отобразятся в дисплее (рис. 4.1). Скопируйте модель в отчет и сделайте выводы по проделанной работе.

3.3. Методические указания по выполнению задания 2

Векторные и матричные операторы и функции системы MATLAB позволяют решать целый ряд задач линейной алгебры. Для реализации различных алгоритмов решения СЛУ и связанных с ними матричных операций применяются следующие матричные операторы: $+$, $-$, $*$, $/$, $\backslash$, $^$, $'$.

MATLAB имеет два типа арифметических операций:

- поэлементные и
- для массивов (векторов и матриц) в целом.

Матричные арифметические операции определяются прави-

лами линейной алгебры. Алгебраические операции сложения и вычитания над массивами выполняются поэлементно. Знак «.» отличает операции над элементами массивов от матричных операций. Поскольку операции сложения и вычитания одинаковы для матрицы и элементов массива, знаки «+» и «-» не используются. Для операторов умножения и деления знак «.» имеет значение. Без точки «*» «/» или «\» – это матричное умножение и деление (правое и левое) соответственно, а с точкой «.*» «./» или «.\» – по элементное умножение и деление.

Например,

$C=A*B$ – линейное алгебраическое произведение матриц A и B :

$$C(i, j) = \sum_{k=1}^n A(i, k) * B(k, j)$$

Для не скалярных A и B число столбцов матрицы A должно равняться числу строк матрицы B . Скаляр может умножаться на матрицу любого размера. Если A – матрица размерами $[n \times n]$, а B – вектор-столбец с n компонентами, то $X=A \setminus B$ – решение уравнения вида $A \cdot X=B$, которое находится методом исключения Гаусса.

Поскольку **вектор B в командной строке представлен в виде строки**, то **матрицу A надо транспонировать**. Решением СЛУ будет $X=B/A'$, где A' – транспонированная матрица A .

1. Для решения системы линейных уравнений матричным способом в системе MATLAB необходимо в режиме прямых вычислений выполнить следующее:

2. Задать матрицу A коэффициентов системы;

3. Задать вектор B свободных членов системы;

4. Вывести решение СЛУ матричным способом:

- $X1=B/A'$ % матричные операторы,
- $X2=B*A'^{-1}$ % использование обратной матрицы,
- $X3=B*inv(A')$ % использование функции $inv()$.

5. Для решения СЛУ методом наименьших квадратов надо задать матрицу A и вектор B (п.2 и 3), а затем вывести результаты решения СЛУ с помощью функции $lsqr(A,B')$. Эта функция возвращает точное решение x , если оно существует, в противном случае – возвращает решение по итерационному методу наименьших квадратов.

Рассмотрим пример решения СЛУ вида:

$$\begin{cases} 4 \cdot x_1 + 0,24 \cdot x_2 - 0,08 \cdot x_3 = 8 \\ 0,09 \cdot x_1 + 3 \cdot x_2 - 0,15 \cdot x_3 = 9 \\ 0,004 \cdot x_1 - 0,08 \cdot x_2 + 4 \cdot x_3 = 20 \end{cases}$$

```
>> % Решение СЛУ
>> A=[4 0.24 -0.08; 0.09 3 -0.15; 0.004 -0.08 4] % коэффициенты
A =
    4.0000    0.2400   -0.0800
    0.0900    3.0000   -0.1500
    0.0040   -0.0800    4.0000
>> B=[8 9 20]
B =
     8     9    20
% матричным методом с помощью оператора «/»
>> X=B/A'
X =
    1.9095    3.1958    5.0620
% с помощью обратной матрицы
>> X1=B*A'^-1
X1 =
    1.9095    3.1958    5.0620
% с помощью функции inv() – обратной матрицы
>> X3=B*inv(A')
X3 =
    1.9095    3.1958    5.0620
% методом наименьших квадратов
>> lsqr(A,B')
ans =
    1.9095
    3.1958
    5.0620
>> % Проверка решения СЛУ
>> X*A'
ans =
    8.0000    9.0000   20.0000
```

Как видно, все методы решения дали один и тот же результат.

6. В *Simulink* создайте модель устройства (рис. 3.3), выполняющего решение СЛУ. В таблице 3.4 указаны названия блоков, папки в которых находятся нужные блоки для моделирования и параметры блоков, которые нужно изменить:

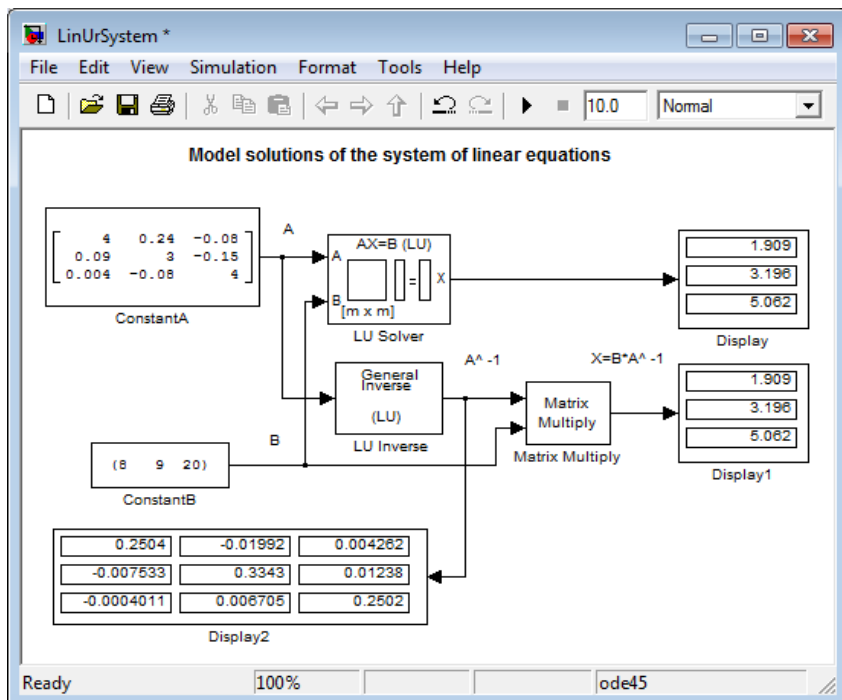


Рис. 3.3. Модель решения системы линейных уравнений (СЛУ)

Таблица 3.4. Значения параметров блоков моделирования СЛУ

Название блоков	Размещение в браузере	Параметры блоков
<i>ConstantA</i> , <i>ConstantB</i>	<i>Simulink / Sources</i>	$[4 \ 0.24 \ -0.08; \ 0.09 \ 3 \ -0.15; \ 0.004 \ -0.08 \ 4]$ $[8 \ 9 \ 20]$ // по варианту
Блок <i>LU Solver</i>	<i>Signal Processing Blockset / Math Function/ Matrices and Linear Algebra / Linear System Solvers</i>	По умолчанию

Блок <i>LU Inverse</i>	<i>Signal Processing Blockset /Math Function/ Matrices and Linear Algebra /Matrix Inverses</i>	По умолчанию
Блок <i>Matrix Multiply</i>	<i>Signal Processing Blockset /Math Function/ Matrices and Linear Algebra /Matrix Operations</i>	По умолчанию
<i>Display,1,2</i>	<i>Simulink/Sinks</i>	По умолчанию

Блоки матричных и математических операций широко представлены в пакете *Signal Processing Blockset* (в старых версиях *Digital Signal Processing Blockset*). *MATLAB* решение $A \cdot X = B$ выполняется методом *LU* разложения. Матрица A должна быть квадратной, вектор B должен иметь одинаковое число строк с A .

При выполнении прямого хода преобразований строк матрицы A матрица приводится к верхнему треугольному виду U (*upper* – верхний) – все числа под диагональю равны нулю. Если при этом не делается перестановка строк, то матрица, умножение на которую слева равносильно сделанным преобразованиям строк, оказывается нижней треугольной L (*lower* – нижний) – все числа над диагональю равны нулю. В результате получаем *LU* разложение матрицы, например:

$$A = LU$$

Функция *lu(A)* выражает любую квадратную матрицу A как произведение двух треугольных матриц, одна из которых нижняя треугольная матрица, а другая – верхняя треугольная матрица. Эту операцию называют *LU*-разложением.

Например, для заданной матрицы A :

```
>> A=[4 0.24 -0.08; 0.09 3 -0.15; 0.004 -0.08 4];
>> A
A =
    4.0000    0.2400   -0.0800
    0.0900    3.0000   -0.1500
    0.0040   -0.0800    4.0000
>> [L,U]=lu(A) % формат записи функции
```

```

L =
    1.0000    0    0
    0.0225    1.0000    0
    0.0010   -0.0268    1.0000
U =
    4.0000    0.2400   -0.0800
         0    2.9946   -0.1482
         0         0    3.9961
>> L*U % исходная матрица A
ans =
    4.0000    0.2400   -0.0800
    0.0900    3.0000   -0.1500
    0.0040   -0.0800    4.0000

```

Блок *LU Solver* (рис. 3.4) – решает линейную систему вида $A \cdot X = B$, применяя *LU* разложение матрицы ($LUX=B$) и имеет два входа – для ввода матрицы A и вектора B . Выходной порт X – это решение СЛУ. Матрица A должна быть квадратной, а вектор B – представлен столбцом с одинаковым число строк, как и у A .

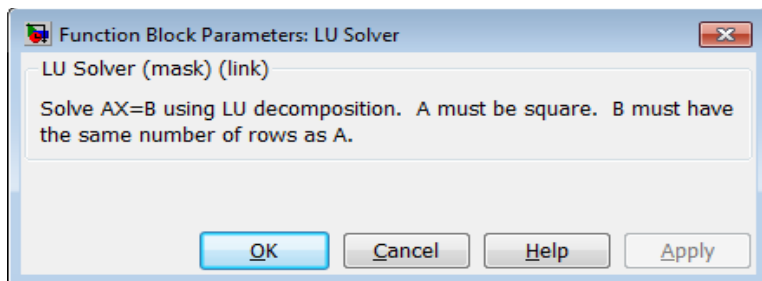


Рис. 3.4. Окно установки параметров блока *LU Solver*

Блок *LU Inverse* (рис. 3.5) – выполняет обратное преобразование матрицы, применяя *LU* разложение матрицы ($A^{-1}=LU^{-1}$), где L -нижняя треугольная квадратная матрица с диагональными элементами, а U -верхняя треугольная квадратная матрица. Блок имеет выход матрицы обратной матрице A^{-1} .

```

>> A^-1
ans =
    0.2504   -0.0199    0.0043
   -0.0075    0.3343    0.0124
   -0.0004    0.0067    0.2502

```

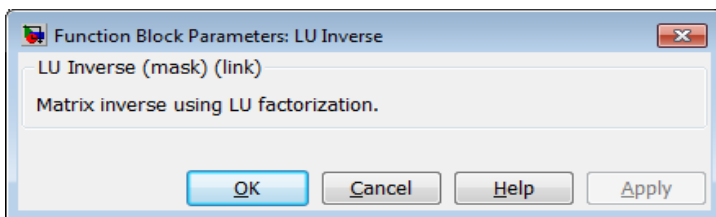


Рис. 3.5. Окно установки параметров блока *LU Inverse*

Блок *Matrix Multiply* выводит результат умножения (или деления) двух (и более) переменных, как скалярных, так и не скалярных. В последнем случае не скалярные величины должны иметь одинаковые размеры.

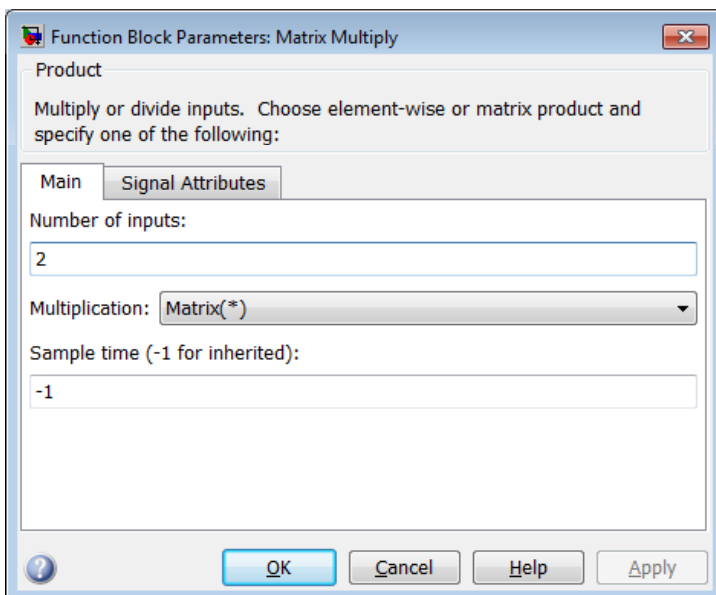


Рис. 3.6. Окно установки параметров блока *Matrix Multiply*

7. Сохраните модель и запустите ее исполнение. Результаты вычислений отобразятся в дисплее (рис. 3.4). Скопируйте модель в отчет и сделайте выводы по проделанной работе.

Как показала проверка моделирования СЛУ путем решения системы с помощью встроенных средств *MATLAB*, модель собрана и функционирует верно.

Пример контрольного теста «Линейные уравнения»

Матричным оператором в системе *MATLAB* не является:

$\wedge$ * ./ .\ -

Матричным оператором в системе *MATLAB* не является:

' \ *. .* /

Укажите правильное задание матрицы В размером 3х3 в *MATLAB*:

$B=[2\ 3\ 4, -5\ 6\ 4, 0\ -7\ 1]$

$B=[2\ 3\ 4; -5\ 6\ 4; 0\ -7\ 1]$

$B=[2, 3, 4, -5, 6, 4, 0, -7, 1]$

$B=[2: 3: 4; -5: 6: 4; 0: -7: 1]$

$B=(2\ 3\ 4; -5\ 6\ 4; 0\ -7\ 1)$

Вывод на экран вектор А в виде столбца:

$A(')$ $A=$ $A(..)$ $A(:)$ $A.^$

Транспонирование массива F

$F.'$ $F\ t$ $F.t$ $F^{\wedge}$ $F.^{\wedge}$

Если А – матрица коэффициентов при неизвестных, а В – вектор свободных членов в системе линейных уравнений, то к матричным способам решения системы относятся:

$X=B/A'$

$X=B\backslash A$

$X=B*A'^{-1}$

$X=B^{-1}*A$

$X=A*B$

Укажите формулу, проверяющую в матричном виде правильность решения СЛУ:

$X*B$

X/A

$X*A'$

X/B

$A'*B$

где А – матрица коэффициентов, В – вектор свободных членов, X – вектор корней решения СЛУ.

4. Решение нелинейных уравнений

4.1. Подготовка к работе

По указанной литературе изучить:

- создание и редактирование графиков в *MATLAB*;
- объекты подбиблиотеки *Math Operations*, *Sinks*, *Sources* библиотеки *Simulink*;
- способы решения нелинейных уравнений в *MATLAB*.

4.2. Задание и порядок выполнения работы

1. Задача 1. Решить два нелинейных уравнения (НУ) с последующим их моделированием. Функции для решения уравнений заданы по варианту в таблице 4.1. Для этого:

- построить графики функций $F1(X)$ и $F2(X)$ в одном окне, определить точки пересечения графиков с осью X ;
- вычислить корни нелинейных уравнений вида $F1(X)=0$ и $F2(X)=0$ с помощью встроенных функций *roots* и *solve*;
- создать модели решения НУ с использованием блоков математических операций пакета *Simulink*.

Таблица 4.1. Варианты заданий

N	Функции		Диапазон изменения x		
	$F1$	$F2$	x_0	x_n	$h(шаг)$
1	$-0.6x^3 - 3x^2 + 10x + 5$	$2x^2 - x - 35$	-10	5	0.3
2	$-0.7x^3 + 2x^2 + 4x - 1$	$-x^2 + \sqrt{ 2x } + 10$	-5	5	0.2
3	$-0.4x^3 - 2x^2 + 2x + 9$	$-3x^2 - x + 2 + 40$	-10	5	0.4
4	$-0.2x^3 - 2x^2 + 2x + 20$	$2x^2 - \sqrt{ x - 1 } - 30$	-12	5	0.5
5	$0.3x^3 + 2x^2 - 6x - 10$	$-6x^2 - x + 25$	-10	5	0.3
6	$-1.1x^3 + 5x^2 + 1x - 15$	$-2x^2 + \sqrt{ x + 1 } + 20$	-5	5	0.2
7	$-0.25x^3 + 3x^2 + 2x - 20$	$3(x + 1)^2 - x - 50$	-10	15	0.3
8	$0.2x^3 - 3x^2 + 5x + 25$	$-2x^2 - \sqrt{ x } + 45$	-10	15	0.5
9	$-0.9x^3 - 3x^2 + 5x + 10$	$2(x - 1)^2 + x - 25$	-5	5	0.3
10	$-0.9x^3 - 4x^2 + 2x + 10$	$2x^2 + \sqrt{ x } - 25$	-5	5	0.2
11	$-0.5x^3 - 3x^2 + 11x + 6$	$3x^2 - 2x - 41$	-10	5	0.3
12	$0.3x^3 - 4x^2 + 3x + 50$	$-3x^2 - \sqrt{ x - 5 } + 50$	-10	15	0.4
13	$-0.4x^3 - 2x^2 + 2x + 9$	$-3x^2 - x + 2 + 40$	-10	5	0.4
14	$-1.1x^3 + 5x^2 + 1x - 15$	$-2x^2 + \sqrt{ x + 1 } + 20$	-5	5	0.2
15	$-0.9x^3 - 3x^2 + 5x + 10$	$2(x - 1)^2 + x - 25$	-5	5	0.3
16	$0.3x^3 - 4x^2 + 3x + 50$	$-3x^2 - \sqrt{ x - 5 } + 50$	-10	15	0.4

- сохранить модели и сравнить результаты моделирования НУ с вычисленными значениями;
- скопировать в отчет графики функций, модели решения НУ и результаты проверки решения.

2. **Задача 2.** Решить **систему** нелинейных уравнений (СНУ) с последующим ее моделированием. Функции для решения системы нелинейных уравнений (СНУ), заданы в табл. 4.1. Для этого:

- определить по графику функций $F1(X)$ и $F2(X)$ окрестности существования корней СНУ;
- решить СНУ вида $(F1(X)-F2(X)=0)$ с помощью встроенной функции `fzero()`;
- создать модель решения СНУ с использованием блоков пакета *Simulink*;
- сохранить модель и сравнить результаты моделирования СНУ с вычисленными значениями;
- скопировать в отчет модель решения СНУ и результаты проверки решения.

3. Сделать выводы относительно величины погрешности вычислений нелинейных уравнений разными методами.

4.3. Методические указания по выполнению задания 1

Довольно часто на практике возникает задача решения (нахождения корней) нелинейного уравнения вида $F(X)=0$ или системы СНУ $F1(X)=F2(X)$. Последнее можно свести к виду $F1(X)-F2(X)=0$. Таким образом, данная задача сводится к нахождению значений аргумента X функции $F(X)$ одной переменной, при котором значение функции равно нулю.

В качестве примера рассмотрим способы решение следующих нелинейных уравнений:

$$-0.5 \cdot x^3 - 7 \cdot x^2 + 5x + 50 = 0 \quad \text{и} \quad 3 \cdot x^2 - \sqrt{|x|} - 100 = 0$$

Как видно первое нелинейное уравнение полиномиальное, второе – степенное (с целочисленными показателями степени).

1. Для решения нелинейного уравнения в графической интерпретации необходимо:

- задать ранжированную переменную X ,
- задать функцию пользователя,

- вызвать окно для отображения графика,
- задать команду построения графика.

Построим графики функций $F1$ и $F2$ в заданном интервале, например, $x \in -15..+10$ (рис. 4.1):

```
>> X=-15:0.5:10;    % Задание ранжированной переменной
>> F1=-0.5*X.^3-7*X.^2+5*X+50;
>> F2=3*X.^2-sqrt(abs(X))-100;
>> figure(1);
>> plot(X,F1,X,F2);
```

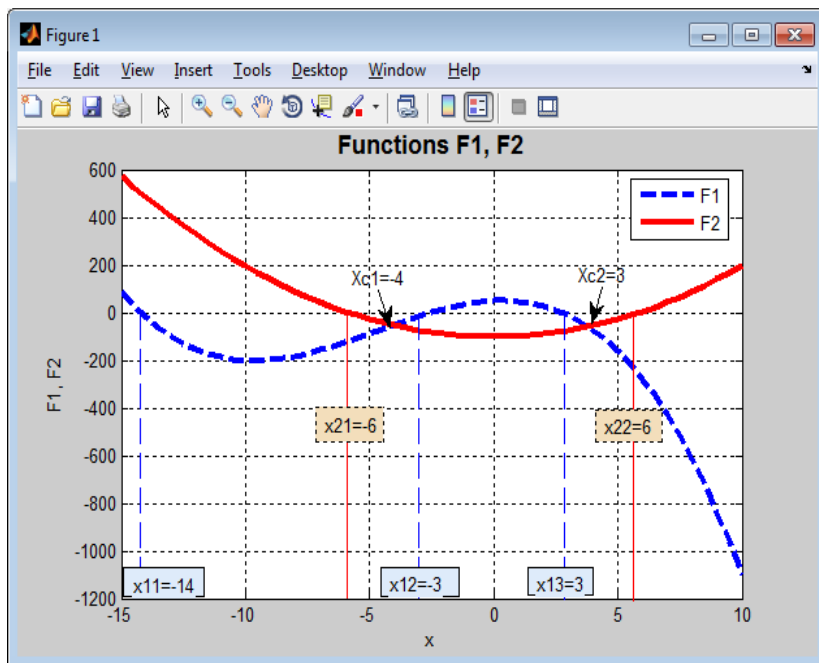


Рис. 4.1. Графики функций $F1$ и $F2$

Как видно из рисунка 4.1 обе функции пересекают ось x в нескольких точках, где и существуют корни уравнений вида $F1(X)=0$ и $F2(X)=0$. Функция $F1$ (отображается на графике пунктирной линией) – полином третьей степени – имеет 3 точки пересечения с осью x в окрестностях точек -14, -3, 2. Вторая функция $F2$ (отображается на графике сплошной линией) – степенная – имеет 2 точки пересечения с осью x в окрестностях -6 и +6.

2. Для решения нелинейных уравнений в режиме прямых вычислений в MATLAB имеются встроенные функции. Так, например, для полинома – функция `roots(F1)`, которая возвращает вектор-столбец X , элементы которого являются корнями заданного полинома. Вектор-строка `F1` содержит коэффициенты полинома, упорядоченные по убыванию степеней. Например, для заданного полинома `F1`:

```
>> % Решение полинома с помощью функции roots():
>> F1=[-0.5, -7, 5, 50];
>> X=roots(F1)
X =
-14.2085
 2.7592
-2.5507
>> % Проверка решения подстановкой, например, второго корня
x=2.7592
>> F=-0.5*X(2)^3-7*X(2)^2+5*X(2)+50
F =
0
```

Как видно корень $x=2.7592$ вычислен точно. Аналогично можно проверить точность вычисления других корней ($x=-14.2085$ и $x=-2.5507$) путем их подстановки в решение нелинейного уравнения.

3. Как известно, для решения нелинейного уравнения в MATLAB имеется функция `solve()`, которая возвращает вектор корней НУ, заданного в символьном виде. Например, для функций $F1$, $F2$:

```
>> % Решение НУ в символьном виде:
>> X1= solve('0.5*x^3-7*x^2+5*x+50')
X1 =
-14.208463160937381019680273693768
-2.550749225446045742603419977223
 2.759212386383426762283693670992
>> X2=solve('3*x^2-sqrt(abs(x))-100')
X2 =
-5.8428646462036777884773935421505
 5.8428646462036777884773935421505
```

[illegible]

Как видно корни вычислены с очень высокой степенью точностью, т.к. при их подстановке $F_2 \rightarrow 0$.

В случае, если в версии MATLAB отсутствует функция `solve()`, можно воспользоваться встроенной функцией `inline()` для задания функции пользователя и функцией `fzero(f,x)`, которая находит точные корни уравнения в окрестности их существования (находятся по графику рис. 4.1). Например,

```
>> F2=inline('3*x^2-sqrt(abs(x))-100')
```

$$F2 =$$

Inline function:

$$F2(x) = 3 \cdot x^2 - \sqrt{\text{abs}(x)} - 100$$

```
>> x1=fzero(F2,-5)
```

$$x_1 =$$

-5.8429

```
>> x2=fzero(F2,6)
```

 $x_2 =$

5.8429

4. В Simulink создайте модель устройства (рис. 4.2), выполняющего решение НУ $F_1=0$ и $F_2=0$. В таблице 4.2 указаны названия блоков, папки в которых находятся нужные блоки для моделирования и параметры блоков, которые нужно изменить при моделировании задачи.

Таблица 4.2. Значения параметров блоков моделирования

Название блоков	Размещение в браузере	Параметры блоков
Блоки <i>Constant1,2</i>	<i>Simulink / Sources</i>	<i>Constant1=3, //устанавливаются Constant2=100 // по варианту</i>
Блок <i>Polynomial</i>	<i>Simulink / Math Operations</i>	<i>Polynomial Coefficient: [-0.5 -7 5 50] // по варианту</i>
Блок <i>Algebraic Constraint1</i>	<i>Simulink / Math Operations</i>	<i>Initial guess 1: [-14 -3 3] //вектор приближенных корней, которые выявляются по графику функций F1(X)</i>

Блок <i>Algebraic Constraint2</i>	<i>Simulink / Math Operations</i>	<i>Initial guess 2: [-6 6] //вектор приближенных корней, которые выявляются по графику функции F2(X)</i>
Блоки <i>Add, Product</i>	<i>Simulink / Math Operations</i>	<i>List of signs: - - + // Add Number of inputs: 2 //Product</i>
Блок <i>Abs</i>	<i>Simulink / Math Operations</i>	По умолчанию
Блоки <i>Math Function1,2</i>	<i>Simulink / Math Operations</i>	Значение <i>Function</i> выбирается из списка функций по варианту
<i>Display1,2</i>	<i>Simulink / Sinks</i>	По умолчанию

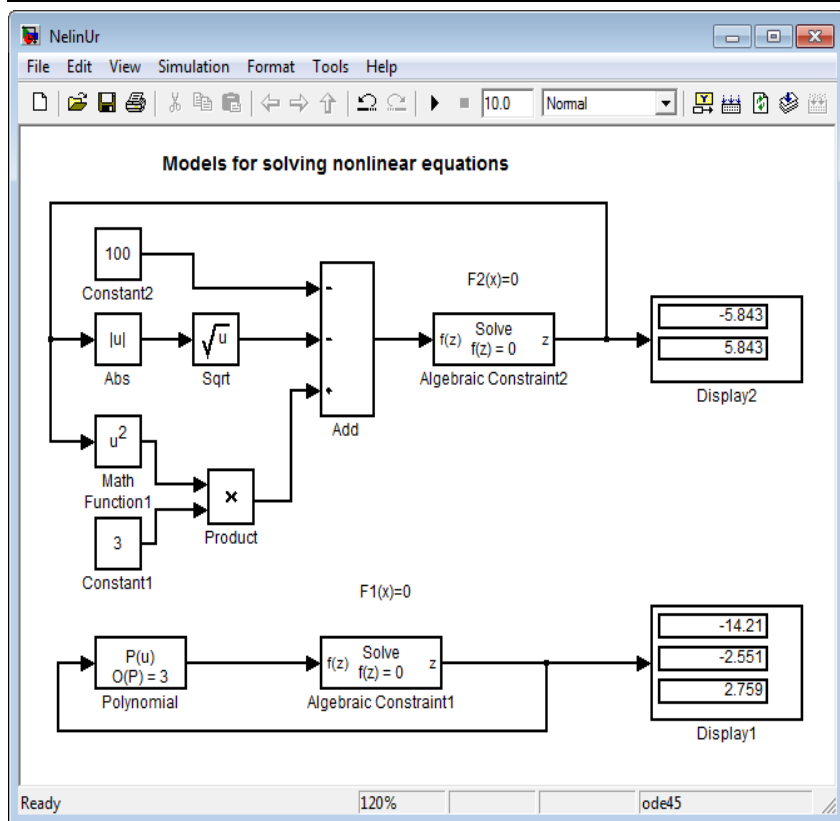


Рис. 4.2. Модель решения НУ.

Пакет расширения *Simulink* имеет обширные возможности в проведении математических операций. Они сосредоточены в разделе их библиотек *Math Function (Math Operations)*. Для решения НУ, рассмотрим некоторые блоки этой библиотеки.

Блок вычисления элементарных функций *Math Function* (математическая функция), *Trigonometric Function* (тригонометрическая функция) и т.п. Действия этих функций и установка их параметров очевидна. Выбор функции осуществляется в раскрывающемся списке окна параметров блока *Math Function* (рис. 4.3).

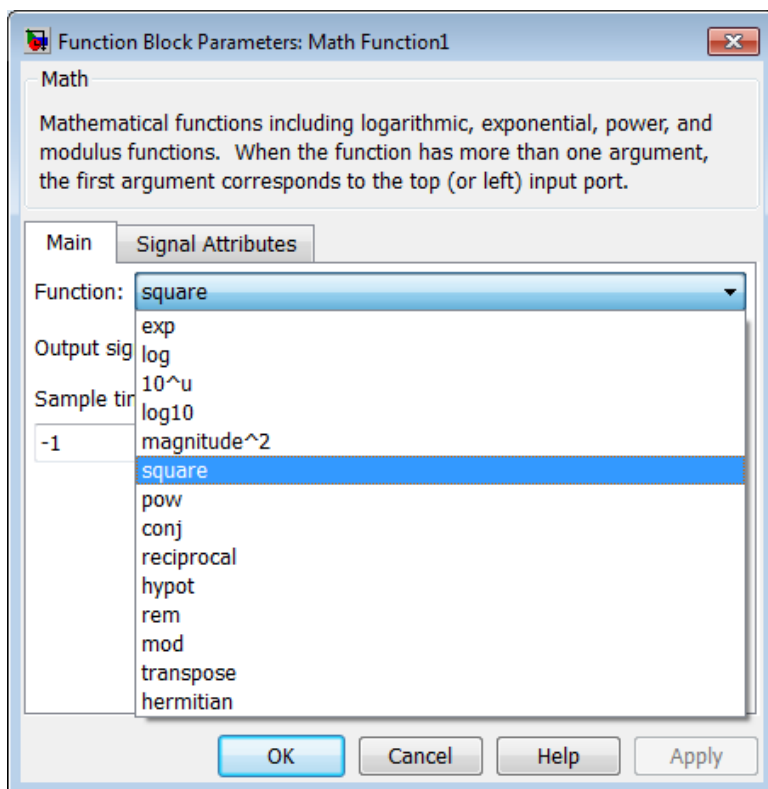


Рис. 4.3. Параметры блока Math Function

Блок алгебраического ограничения *Algebraic Constraint* (рис. 4.4) служит для решения уравнений или систем уравнений. При этом входной сигнал должен быть связан с выходным сиг-

налом (см. рис. 4.2). Используя свойства блока, целесообразно задать начальное приближение корней уравнения для повышения эффективности решателя. Например, вектор $[-14 \ -3 \ 3]$.

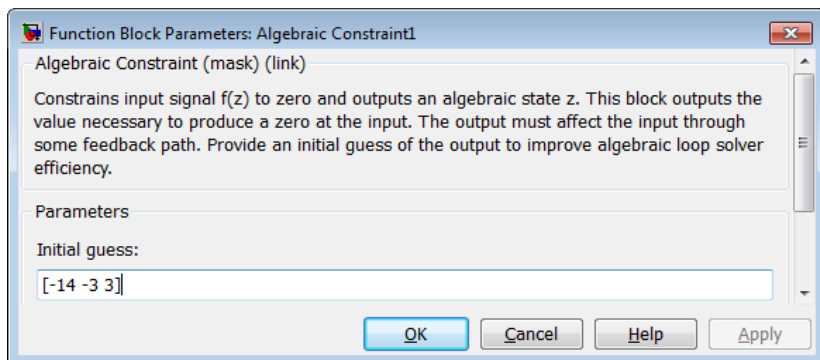


Рис. 4.4. Параметры блока *Algebraic Constraint*

Блок *Polynomial* (рис. 4.5) реализует зависимость, задаваемую степенным многочленом. Сами же коэффициенты полинома в векторном виде должны располагаться по убыванию степени переменной. Например, для полинома $-0.5 \cdot x^3 - 7 \cdot x^2 + 5x + 50$ необходимо задать вектор коэффициентов $[-0.5 \ -7 \ 5 \ 50]$. Коэффициенты должны быть действительного типа.

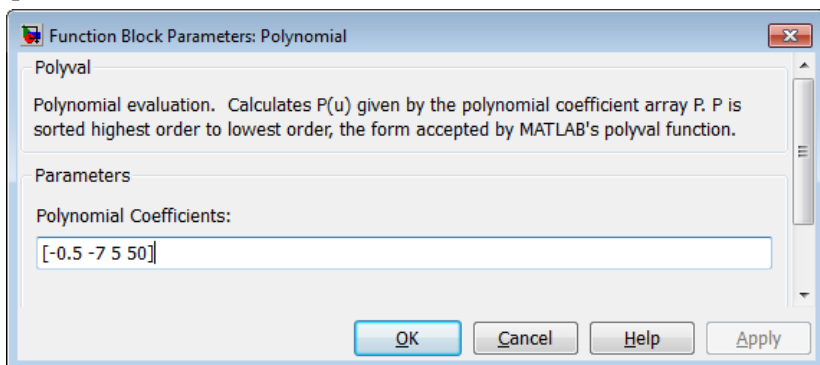


Рис. 4.5. Параметры блока *Polynomial*

Блоки *Add*, *Subtract*, *Sum*, *Product* – выполняют арифметические действия над операндами в соответствии со своим названием. В окне параметров блоков устанавливаются количество входов, а в блоках сложения и вычитания – еще и знак операнда.

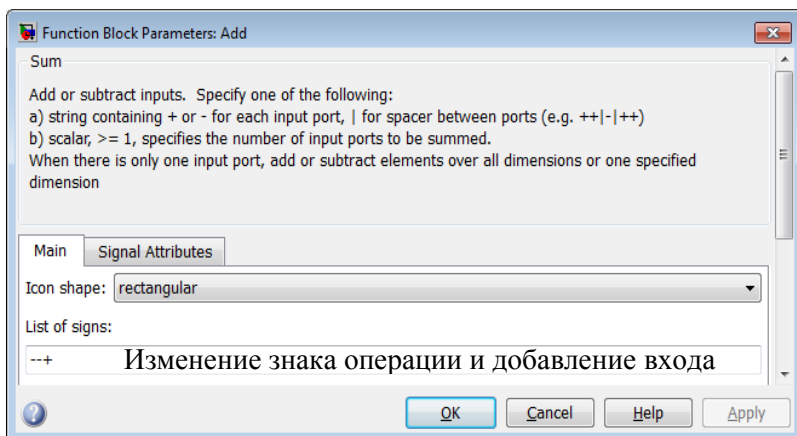


Рис. 4.6. Параметры блока Add

5. Сохраните модель и запустите ее исполнение. Результаты решений $F1=0$ и $F2=0$ соответственно отобразятся на дисплеях (рис. 4.2). Скопируйте модель в отчет и сделайте выводы по проделанной работе.

Как показала проверка моделирования НУ путем решения с помощью встроенных средств *MATLAB*, модель собрана и функционирует верно.

4.4. Методические указания по выполнению задания 2

Решение СНУ сводится к виду $F1(x)-F2(x)=0$. Таким образом, необходимо найти значения аргумента x , когда функции $F1(x)=F2(x)$ равны друг другу.

1. Для решения системы нелинейных уравнений в графической интерпретации (рис. 4.1) необходимо построить график и найти корни как окрестности пересечения функций между собой в заданном интервале $x \in -15..+10$. Для рассматриваемых уравнений $-0.5 \cdot x^3 - 7 \cdot x^2 + 5x + 50$ и $3 \cdot x^2 - \sqrt{|x|} - 100$ — это $x = -4$ и $x = 3$.

2. Для решения СНУ в командной строке *MATLAB* можно использовать встроенную функцию *inline()* и функцию *fzero(f,x)*, которая находит точные корни уравнений. Функция *fzero(f[x1 x2])* возвращает значение x , при котором $f(x)=0$

с заданием интервала поиска с помощью вектора $x = [x_1 \ x_2]$, такого, что знак $f(x(1))$ отличается от знака $f(x(2))$. Возвращенное значение близко к точке, где функция меняет знак. Иначе выдается сообщение об ошибке. Например:

```
>> % Решение системы нелинейных уравнений (СНУ)  $f=F_1-F_2$ :
>> f=inline('-0.5*x^3-7*x^2+5*x+50-3*x^2+sqrt(abs(x))+100')
f =
    Inline function:
     $f(x) = -0.5*x^3-7*x^2+5*x+50-3*x^2+sqrt(abs(x))+100$ 
>> x1=fzero(f,-4) % в окрестности существования первого корня
x1 =
    -4.0656
>> f(x1) % решение СНУ при  $x=-4.0656$ 
ans =
    1.4211e-014
>> x2=fzero(f,3) % в окрестности существования второго корня
x2 =
    3.7904
>> f(x2) % проверка решения СНУ при  $x=3.7904$ 
ans =
    1.4211e-014
```

Как видно, по результату решения $f=F_1-F_2 \rightarrow 0$. Корни СНУ вычислены с некоторой погрешностью ($1.4211e-014$), которая сказывается при проверке решения СНУ функции f .

3. Для создания модели решения СНУ достаточно скопировать предыдущие модели в новое окно и отредактировать их, выполнив операцию алгебраического вычитания $f=F_1-F_2$. Полученная модель решения СНУ для рассматриваемых функций приведена на рис. 4.7.

Согласно графикам функций F_1 и F_2 , приведенным на рис. 4.1, корни СНУ находятся в окрестности точек $[-4 \ 3]$. Эти значения в векторном виде надо ввести в блок *Algebraic Constraint1*, изменив при этом первоначальные значения, а блоки *Algebraic Constraint2* и *Consonant2* удалить. Кроме того, в модель надо ввести блок вычитания *Subtract*, который позволит реализовать

операцию $F1-F2$.

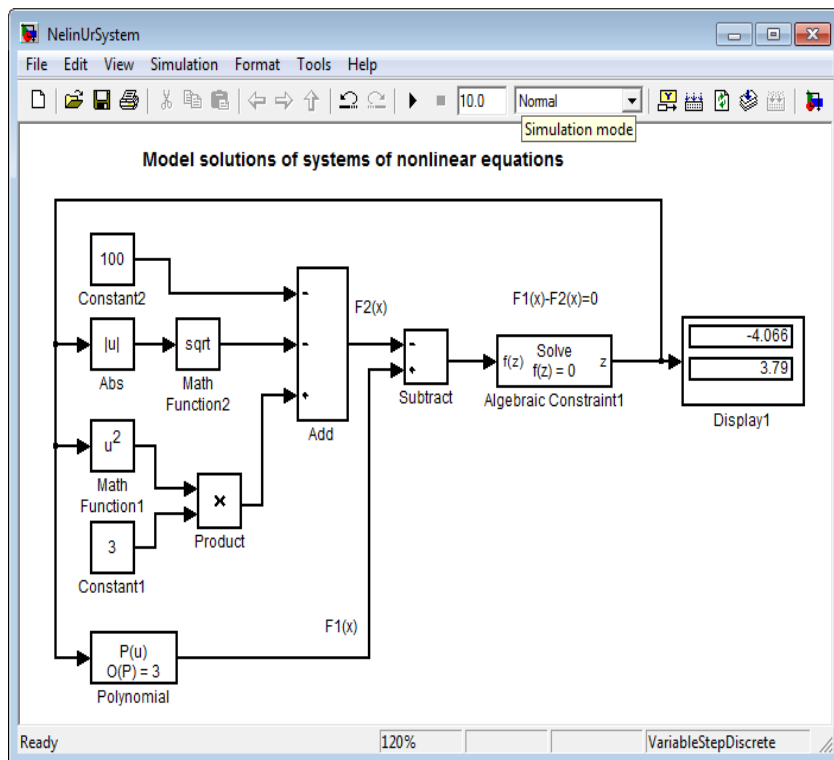


Рис. 4.7. Модель решения СНУ.

4. Сохраните модель и запустите ее исполнение. Результаты решения $F1-F2=0$ соответственно отобразятся на дисплее (рис. 4.7). Скопируйте модель в отчет и сделайте выводы по проделанной работе.

Как показала проверка моделирования СНУ путем решения с помощью встроенных средств *MATLAB*, модель собрана и функционирует верно.

Работоспособность всех моделей проверена с помощью встроенных средств системы *MATLAB*. Все они работоспособны и выполняют поставленные задачи.

Пример контрольного теста «Нелинейные уравнения»

Укажите оператор формирования упорядоченных последовательностей в диапазоне от А до В с шагом Н в MATLAB:

X=A, A+H..B

A, H.. B

A: H: B

A: A+H: B

A..A+H..B

Укажите функцию, возвращающую точные значения корней полинома в системе MATLAB:

poly()

roots()

polyroots()

root()

sqrt()

Укажите функцию, возвращающую точные значения корней нелинейного уравнения в системе MATLAB:

root()

solve()

polyroots()

zero()

Укажите правильный вариант задания вектора – полинома:

Y=[a0,a1,a2,a3]

Y=[a3,a2,a1,a0]

Y=(a3,a2,a1,a0)

Y=(a0,a1,a2,a3)

Y=[a3.a2.a1.a0]

Укажите функцию, возвращающую точные значения корней системы нелинейных уравнений в системе MATLAB:

roots()

solve()

fzero()

polyroots()

zero()

5. Программирование в MATLAB

5.1. Подготовка к работе

По указанной литературе изучить:

- средства и особенности программирования MATLAB;
- структура и свойства файлов сценариев;
- структура и свойства файлов функций.

5.2. Задание и порядок выполнения работы

1. Запустить систему MATLAB и ввести текстовый комментарий "М-файлы сценарии и функции".
2. Задача 1. По заданным расчетным формулам и наборам исходных данных, указанных в табл.5.1 вычислить значения функции. Построить график функции с шагом 0,2. Решение привести в MATLAB с использованием файла сценария.

Таблица 5.1. Варианты заданий.

N	Функция	xmin	xmax	a	b
1	$y = \begin{cases} \sin bx^2, & \text{при } x < 0 \\ x \cos^3 x, & \text{при } x > 3 \end{cases}$	-2	5	-	2
2	$c = \begin{cases} b \cos x, & \text{при } x < 2; \\ x\sqrt{x+a}, & \text{при } 2 \leq x < 4 \end{cases}$	0	6	1,5	1,5
3	$p = \begin{cases} tg(x+ax^2), & \text{при } 1 < x \leq 3 \\ 2 \cos(x+b), & \text{при } x = 1 \end{cases}$	0	3	2	3
4	$y = \begin{cases} \sqrt{ x } - \cos x, & \text{при } x < 0 \\ \cos^2 ax, & \text{при } 0 \leq x \leq 2 \end{cases}$	-2	3	0,5	-
5	$y = \begin{cases} x + atgx, & \text{при } x = 3 \\ x \cos x^2, & \text{при } x > 3 \end{cases}$	1	4	0,5	-
6	$z = \begin{cases} x \ln x^2, & \text{при } x > 4 \\ b \cos x, & \text{при } x < 3 \end{cases}$	2	5	-	3

7	$t = \begin{cases} \ln^2(x + a), & \text{нпу } 0 < x < 3 \\ 3\operatorname{tg}x, & \text{нпу } x \geq 3 \end{cases}$	0	5	1,65	-
8	$y = \begin{cases} b \cos^2 x, & \text{нпу } x > 4 \\ a \ln x + \sqrt{x}, & \text{нпу } 2 < x \leq 4 \end{cases}$	2	5	2,5	2,6
9	$t = \begin{cases} a10^x, & \text{нпу } x \geq 2 \\ \sin(\ln x), & \text{нпу } 1 \leq x < 2 \end{cases}$	0,5	3	1,5	-
10	$z = \begin{cases} ax^2 + bx, & \text{нпу } 3 < x < 5 \\ x + \operatorname{tg}x, & \text{нпу } x \leq 3 \end{cases}$	1	5	2	5
11	$p = \begin{cases} b \ln x^2, & \text{нпу } x \geq 4 \\ (x - 2)^2, & \text{нпу } 1 \leq x < 4 \end{cases}$	0	5	-	1,5
12	$f = \begin{cases} a - 3 \ln x, & \text{нпу } x > 3.5 \\ bx - 7/x^2, & \text{нпу } x < 2.5 \end{cases}$	1	5	2,3	-0,3

3. Задача 2. Решить систему нелинейных уравнений F1(x) и F2(x) по варианту, приведенному в табл. 5.2. Для этого:

- создать два М-файла для заданных функций F1(x), F2(x). Сохранить их в своей папке;
- создать М-файл для разностной функции F3(x)=F1(x)-F2(x). Сохранить его в своей папке;
- построить графики этих функций в одном окне, определить окрестности существования корней каждого из уравнений F1(x)=0 и F2(x)=0, как пересечения графиков с осью x и системы уравнений, как координаты пересечения двух графиков F1(x) и F2(x), т.е. функции F3(x).
- используя функцию fzero(), найти точные значения каждого корня. Сформировать строку результатов и вывести ее в заголовке окна.

4. Графики, сценарий и вызов сценария скопировать в отчет, созданный в текстовом редакторе MS Word. Сохранить его в своей папке.

5. Сделать выводы о методах решения нелинейных уравнений и возможностях программирования в MATLAB.

Таблица 5.2. Варианты заданий

N	Расчетные формулы	Данные
1	$s = 1,5 \cos x; \quad w = (y - x) \frac{y}{1 + (y - x)^2}$	y=7; z=-3.3; x=4..9
2	$y = e^{bx} \sin(a \cdot x + b) - \sqrt{ b \cdot x + a }; \quad s = a \sin(x)$	a=-3; b=0.3; x=5..8
3	$w = \sqrt{x^2 + b} - b^2 \sin^3(x + a) / x; \quad y = b \cos \frac{x}{2}$	a=0.5; b=2.1; x=0..5
4	$s = x^3 \sin(x + b) + \frac{a^2}{\sqrt{x + b}}; \quad q = bx^2 - a$	a=6.5; b=2; x=0..4
5	$r = \frac{x(x+1)}{b} - \sin^2(x + a); \quad s = \cos(x + b) \sqrt{\frac{xb}{a}}$	a=1.5; b=8; x=0..5
6	$y = \frac{\sin(x^2 + a)}{b} - \sqrt{\frac{ x }{b}}; \quad z = \frac{x^2}{a} - \cos(x + b)^3$	a=0.1; b=0.02; x=-1..2
7	$f = \sqrt{ c \cdot \sin(x) }; \quad z = m \cos(b \cdot x \sin(x)) + c$	m=2; b=0.7; c=-1; x=0..2
8	$y = a \cdot \sin \frac{x}{a}; \quad d = a \cdot \cos \frac{b^2 \cdot x}{a}$	a=3.2; b=17; x=-0.5..2
9	$f = a \ln(a + x^2) \sin^3 \frac{x}{b}; \quad z = x^2 - a$	a=5; b=1.2; x=1..5
10	$y = b \cos(x^2); \quad r = \sqrt{x^2 + b} - x^3 b^3 \sin(x + a)$	a=0.3; b=0.9; x=-2..0
11	$z = a \cdot x \sin(2x) + e^{-2x} (x + b); \quad t = b \cos x$	a=0.3; b=5; x=0..3
12	$y = 1 - e^{-bx} \sqrt{x + 1.5}; \quad z = \frac{\sin(x)}{\sqrt{1 + \sin^2 x}}$	a=0.2; b=1.4; x=0..4

5.3. Методические указания по выполнению задания

Язык MATLAB является интерпретатором, каждая инструкция распознается и тут же исполняется (что облегчает диалоговый режим). Система MATLAB не создает исполняемых программ в виде машинных кодов (*.exe).

Программы существуют только в виде М-файлов, для исполнения которых нужна среда MATLAB.

М – файлы делятся на два класса:

- файлы **сценарии**, не имеющие входных параметров (процедуры без параметров),
- файлы **функции**, имеющие входные параметры.

Структура файлов сценариев

```
% Основной комментарий           //в первой строке  
% Дополнительный комментарий  
                                //последующие строки
```

Тело файла с любыми выражениями

Свойства файлов сценариев:

- они не имеют входных и выходных параметров;
- работают с данными из рабочей области;
- в процессе выполнения не компилируются;
- представляют собой последовательность операций, аналогичную сессии.

Рассмотрим создание файла сценария на примере вычисления функции $y(x)$, в интервале $-3..5$ с шагом $0,2$, если параметр $b=5$.

$$y(x) = \begin{cases} x^2, & \text{при } x \leq 0 \\ b \sin(x), & \text{при } x > 2 \end{cases}$$

Для создания файла-сценария:

1. Активизировать команду: **File=>New=>Blank M-File**;
2. Открывается окно редактора, в которое вводится программа расчета заданной функции (см. рис. 5.1).

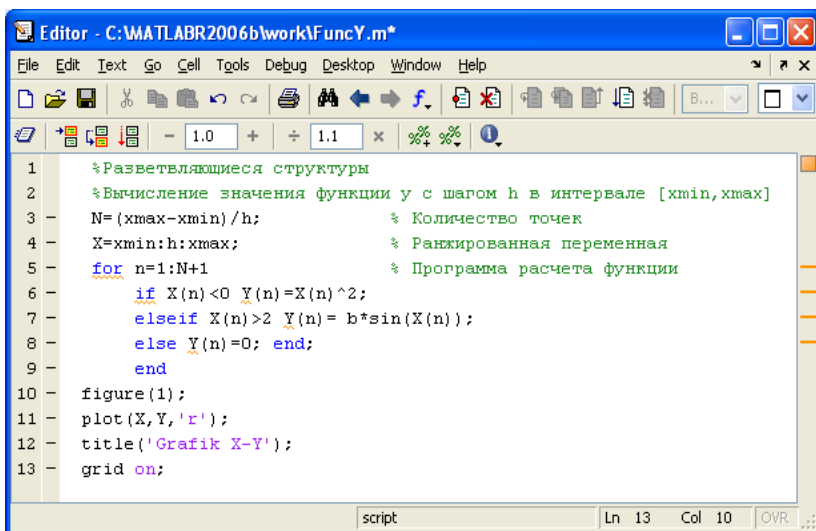


Рис. 5.1. Окно редактора М-файла

3. Сохранить файл в окне Editor командой: **File=>Save as =>FuncY.m.**
4. В окне Command Window необходимо задать конкретные значения (входные параметры) и вызвать файл по имени в командной строке.

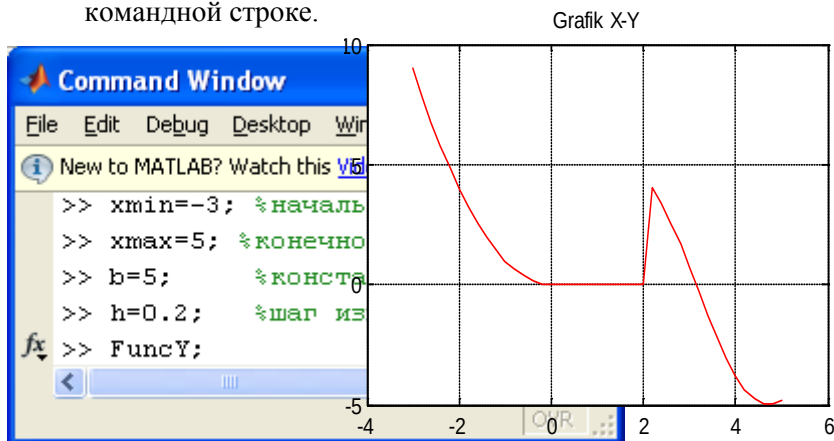


Рис.5.2. Ввод данных, вызов функции Y и ее график.

Структура файлов функций с одним выходным параметром

```
function var=f_name(Список_параметров)
% Основной комментарий
% Дополнительный комментарий
Тело файла с любыми выражениями
var=<выражение>    % вводится, если требуется,
% чтобы функция возвращала результат вычислений.
```

Структура файлов функций с несколькими выходными параметрами:

```
function [var1,var2,...]=f_name(Список_параметров)
% Основной комментарий
% Дополнительный комментарий
Тело файла с любыми выражениями
var1=<выражение>
var2=<выражение>
Такая функция напоминает процедуру.
```

Свойства файлов-функций:

- начинается с объявления **function**, с указанием имя переменной **var** – выходного параметра, имя самой функции и список ее входных параметров;
- функция возвращает свое значение и может использоваться в виде: **name(Список_параметров)** в математических выражениях;
- все переменные в теле файла являются локальными;
- является самостоятельным программным модулем;
- правила вывода комментариев те же, что у файлов сценариев;
- при обнаружении файла функции, он компилируется, и затем исполняется, а созданные машинные коды хранятся в рабочей области системы MATLAB;
- служит средством расширения системы MATLAB.

Статус переменных в функциях

Переменные, указанные в списке параметров функции, являются *локальными* и служат для переноса значений, которые подставляются на их место при вызовах функций. Изменение значений переменных в теле функции не влияет на значения, которые те же переменные могут иметь за пределами функции.

Рассмотрим создание файла функции на примере вычисления функций $y(x)$, $z(x)$ и $t(x)=y(x)-z(x)$ в интервале $-3..3$ с шагом $0,2$, а так же нахождения коней системы нелинейных уравнений $t(x)=y(x)-z(x)$.

$$y = e^{-ax} \cdot (x + 1) + e^{-bx} \cdot (x + 1.5); \quad z = c \cdot \cos(x)$$

Для создания файла-функции:

1. Активизировать команду: **File=>New=>Blank M-File;**
2. В три отдельных окна редактора M-File поочередно ввести программы для расчета функций $F1(x)$, $F2(x)$ и $F3(x)$ с последующим их сохранением командой: **File=>Save as=>**под именами $F1.m$, $F2.m$, $F3.m$ соответственно.

```

1 function Y=F1(x);
2     a=0.2;
3     b=1.5;
4     Y=exp(-a.*x).*(x+1)+exp(-b.*x).*(x+1.5);

```

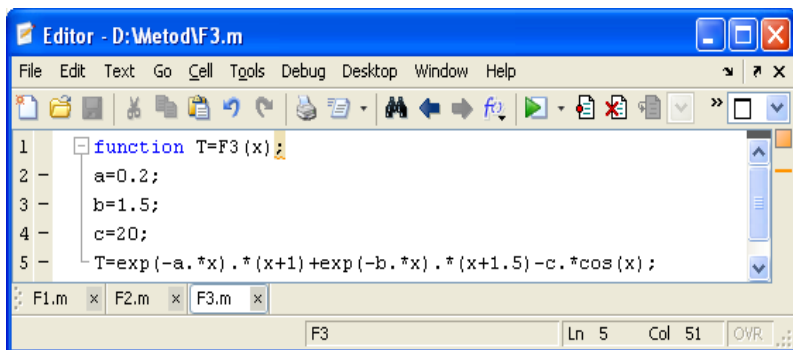
F1.m

```

1 function Z=F2(x);
2     c=20;
3     Z=c.*cos(x);

```

F2.m



F3.m

Рис. 5.3. Окна редактора **М-файлов** функций Y, Z, T.

3. В окне Command Window задаем исходные значения и выводим график F1(X) и F2(X).

```

>> % Исходные данные
>> xmin=-3; % начальное значение
>> xmax=3; % конечное значение
>> h=0.2; % шаг аргумента
>> X=xmin:h:xmax; % вектор аргумента
>> % График локализации корней
>> plot(X,F1(X),'-b*',X,F2(X),'m. '); grid on;

```

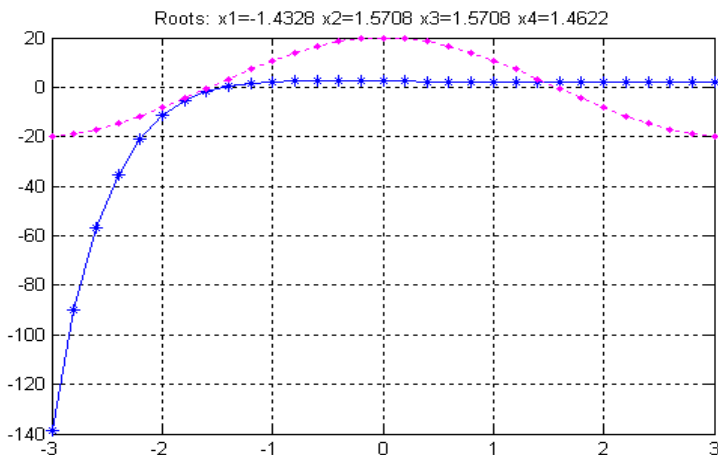


Рис.5.4. Вывод заданных функций и корней уравнений

4. Анализируем график (рис. 5.4) и находим визуально окрестности существования корней:

- функции $F1(x)$ имеет один корень и ее график пересекает ось x в интервале $-2..-1$,
- функция $F2(x)$ имеет два корня и ее график пересекает ось x в двух интервалах: первый $-2..-1$, второй $1..2$,
- функция $F3(x)$ имеет один корень и один интервал пересечения оси x , который можно принять $1..2$,

5. Вызываем файлы-функции по имени относительно найденных окрестностей существования корней.

```
>> x1= fzero('F1(x)',[-2 -1]);    % корень функции F1(x)
>> x1
x1 =
    -1.4328
>> x2= fzero('F2(x)',[-2 -1]);    % 1 корень функции F2(x)
>> x2
x2 =
    -1.5708
>> x3=fzero('F2(x)',[1 2]);       % 2 корень функции F2(x)
>> x3
x3 =
     1.5708
>> x4=fzero('F3(x)',[1 2]);       % корень системы уравнений
F1(x) и F2(x)
>> x4
x4 =
     1.4622
>> Result=strcat('Roots: x1=',num2str(x1),' x2=',num2str(x2),'
x3=',num2str(x3),' x4=',num2str(x4));
                                % строка результата
>> title(Result);               % заголовок графика
```

Система MATLAB с помощью функции `fzero()` находит точные корни уравнений. Функция `fzero(fun[x1 x2])` возвращает значение x , при котором $\text{fun}(x)=0$ с заданием интервала поиска с помощью вектора $x = [x1 \ x2]$, такого, что знак $\text{fun}(x(1))$ отличается от знака $\text{fun}(x(2))$. Возвращенное значение близко к точке, где функция меняет знак. Иначе выдается сообщение об ошибке.

Пример контрольного теста «Программирование»

Укажите все логические операторы в MATLAB:

* & + | ~

Укажите все операторы сравнения в MATLAB:

== <> >= ~= =

Условие $-1 \leq x < 5$ на языке MATLAB:

$(x < 5)$ or $(x \geq -1)$

$x < 5$ & $x \geq -1$

$(x > -1)$ and $(x \leq 5)$

$x > -1$ | $x < 5$

$(x > -1)$ and $(x < 5)$

Укажите строку кода на языке MATLAB, соответствующую

$$\overbrace{y=e^{-2x}}^1 \quad \overbrace{y=\operatorname{ctg}^2(x)}^5 \quad \overbrace{y=e^{-2x}}^5$$

задаче: 1 5 (X - вектор)

if $X > 5$ | $X < 1$ $Y = \exp(-2 * X)$ else $Y = \sin(X).^2 / \cos(X).^2$; end;

if $X > 5$ | $X < 1$ $Y = \exp(-2 * X)$; else $Y = \cot(X.^2)$; end;

if $X > 1$ & $X < 5$ $Y = \operatorname{sqr}(\cot(X))$; else $Y = \operatorname{sqr}(\exp(-X))$; end;

if $X > 5$ | $X < 1$ $Y = \exp(-2 * X)$; else $Y = \cot(X).^2$; end;

if $(X > 1)$ and $(X < 5)$ then $Y := \operatorname{sqr}(\cos(X) / \sin(X))$ else $Y := \exp(-2 * X)$;

Строка кода на языке MATLAB, соответствующая формуле:

$$t = \begin{cases} x \ln x^2, & \text{if } x > 4; \\ \cos^2 x, & \text{if } x \leq 4. \end{cases} \quad (\text{переменная } X - \text{вектор})$$

if $X \leq 4$ $T = \cos(X.^2)$; else $T = X * \ln(X.^2)$; end;

if $X \leq 4$ $T = \cos(X).^2$; else $T = X * \ln(X.^2)$;

if $X \leq 4$ $T = \cos(X).^2$; else $T = X * \log(X.^2)$; end;

if $X \leq 4$ $T = \cos(X)^2$; else $T = X * \log(X.^2)$; end;

if $X > 4$ $T = \cos(X).^2$; else $T = X * \log(X.^2)$; end;

$$q = \frac{x^2 \lg(|a + x|)}{\cos^3(x)}$$

Введите выражение на языке MATLAB, где

переменные Q и X - векторные величины, а - скалярная.
Последовательность слов, определяющих файл-сценарий

- 1 Запись
- 2 серии команд,
- 3 без входных
- 4 и выходных
- 5 параметров.

Установите порядок следования разделов М-файла сценария

- 1 Основной комментарий
- 2 Дополнительный комментарий
- 3 Тело файла с любым выражением

Установите порядок создания М-файла сценария

- 1 Вызов редактора для создания М-файла;
- 2 Заполнение редактора строками программы;
- 3 Сохранение файла в окне редактора;
- 4 Ввод в командной строке входных данных;
- 5 Вызов по имени М-файла в командной строке.

Укажите 3 свойства М-файла сценария

отсутствие входных и выходных параметров
работает с данными из рабочей области
компилируется в процессе выполнения
вызывается в командной строке по имени
возвращает только одно значение

Последовательность слов, определяющих файл-функцию

- 1 Запись
- 2 серии команд,
- 3 с входными и
- 4 выходными
- 5 параметрами.

Установите порядок следования разделов М-файла функции

- 1 Имя функции со списком параметров
- 2 Основной комментарий
- 3 Дополнительный комментарий

4 Тело файла с любым выражением

Установите порядок создания М-файла функции

1 Вызов редактора для создания М-файла;

2 Заполнение редактора строками программы;

3 Сохранение файла в окне редактора;

4 Ввод в командной строке конкретных входных параметров;

5 Вызов функции по имени в командной строке.

Укажите порядок слов раздела объявления файла функции

1 function

2 имена выходных параметров

3 =

4 имя самой функции

5 список входных параметров

Укажите свойство, не относящееся к М-файлу функции

начинается с объявления функции

возвращает значение и может использоваться в математических выражениях

отсутствие входных и выходных параметров

компилируется, а затем исполняется

переменные в теле файла функции являются локальными

Укажите 3 особенности М-файла функции

невозможность запуска файла без предварительной подготовки

имеет локальный статус переменных

имена функций можно использовать в математических выражениях

возвращает множество результатов - по числу выходных параметров

не является самостоятельным программным модулем

Глобальные имена доступны...

только в командной строке

только в файле-функции

в командном окне и, в том числе, и файле-функции

всегда во всех модулях

все ответы правильны

6. Структуры ветвления

6.1. Подготовка к работе

По указанной литературе изучить:

- средства и особенности программирования MATLAB;
- разветвляющиеся структуры и операторы их реализующие.

6.2. Задание и порядок выполнения работы

1. Запустить систему MATLAB и ввести текстовый комментарий "Управляющие структуры".
2. Задача1. Создать М-файл функцию с одним входным и одним выходным параметром для решения задачи, приведенной в табл. 6.1.

Таблица 6.1. Варианты заданий

N	Задания 1
1	Проверить число X на кратность 3 (трем) с выводом результата проверки сообщением
2	Заданы два числа A и B. Выдать текстовое сообщение, если оба они превышают число C
3	Проверить функцию $f(x)$ на четность с выводом результата проверки
4	Заданы два числа A и B. Если оба они положительные, то рассчитать их сумму, в противном случае их произведение.
5	Выдать соответствующие сообщения, что число N попало в интервалы от -10 до -1 или от 1 до 10
6	Проверить число Z на четность в интервале от -9 до 9 с выводом результата проверки
7	Задано число B. Если оно отрицательное, то вывести его абсолютное значение, в противном случае увеличить в 5 раз
8	Выдать текстовое сообщение, если число d попало в третью декаду 12 месяца
9	Задано число Y. Выдать сообщение "No", если это число не превышает числа A и B
10	Заданы два числа A и B. Если они имеют разные знаки, рассчитать их произведение, иначе вывести их сумму

11	Выдать текстовые сообщения, если число d попало в диапазон от 1 до 5 , или число d равно 6 или 7.
12	Заданы два числа A и B . Если оба они отрицательные, то вывести остаток от их деления, в противном случае их сумму.

3. Задача2. Создать М-файл функцию с несколькими выходными параметрами для решения задачи, приведенной в табл. 6.2.

Таблица 6.2. Варианты заданий

N	Задания
1	Даны x , y . Если x и y отрицательны, то их значения возвести в куб; если отрицательно только одно из них, то оба значения увеличить вдвое; если оба значения неотрицательны, то уменьшить их в 2 раза.
2	Даны 2 разных по значению числа (a и b). Большее из этих двух чисел заменить его квадратом, а меньшее – полусуммой заданных чисел.
3	Даны x , y . Если x и y положительные, то их значения уменьшить втрое; если отрицательно только одно из них, то оба значения уменьшить на 0.5; если оба значения отрицательны, то заменить их абсолютной величиной.
4	Даны 2 разных по значению числа (a и b). Меньшее из этих двух чисел заменить их произведением, а большее – в два раза уменьшить.
5	Даны x , y . Если x и y положительные, то их значения увеличить на 1,5; если отрицательно только одно из них, то оба значения уменьшить вдвое; если оба значения отрицательны, то заменить их абсолютной величиной.
6	Даны 2 разных по значению числа (a и b). Большее из этих двух чисел заменить их полусуммой, а меньшее – его квадратом.
7	Даны x , y . Если x и y положительные, то их значения увеличить вдвое; если отрицательно только одно из них, то оба значения уменьшить на 0,5; если оба значения отрицательны, то возвести их в квадрат.
8	Даны 2 разных по значению числа (a и b). Меньшее из этих двух чисел заменить их полусуммой, а большее - их

	удвоенным произведением.
9	Даны x, y . Если x и y отрицательны, то их значения заменить абсолютной величиной; если отрицательно только одно из них, то оба значения увеличить на 0.5; если оба значения неотрицательны, то увеличить их в 10 раз.
10	Даны 2 разных по значению числа (a и b). Большее из этих двух чисел заменить их суммой, а меньшее - их утроенным произведением.
11	Даны x, y . Если x и y отрицательны, то их значения возвести в квадрат; если отрицательно только одно из них, то оба значения увеличить вдвое; если оба значения неотрицательны, то уменьшить их в 5 раз.
12	Даны 2 разных по значению числа (a и b). Меньшее из этих двух чисел заменить их разностью, а большее - их произведением.

4. Программы и блок-схемы поместить в отчет, созданный в текстовом редакторе MS Word. Сохранить его в своей папке.

5. Сделать выводы о разновидностях разветвляющих структур и форматах записи оператора, реализующего эти структуры в системе MATLAB.

6.3. Методические указания по выполнению задания

Практически любая серьезная программа имеет нелинейную структуру. Для создания таких программ необходимы специальные управляющие структуры.

Для создания условных выражений служит оператор *if*.

Условный оператор *if* имеет несколько форматов записи:

Простейший формат

```
if <условие>
    Инструкции;
end;
```

Полный формат

```
if <условие>
    Инструкции_1;
else Инструкции_2;
end;
```

Вложенный формат

```
If <условие_1>
    Инструкции_1;
elseif <условие_2>;
    Инструкции_2;
else Инструкции_3;
end;
Групп elseif <условие> может быть много.
```

Для множественного ветвления применяется переключатель switch:

Формат оператора

```
switch <Селектор>
case {Список значений}
    Список инструкций_1;
case {Список значений}
    Список инструкций_2;
.....
otherwise
    Список инструкций_N;
end
```

Для создания условных выражений служат операторы отношения:

Оператор	Функция	Описание
$x==y$	eq(x,y)	Равно
$x\neq y$	ne(x,y)	Не равно
$x<y$	lt(x,y)	Меньше чем
$x>y$	gt(x,y)	Больше чем
$x\leq y$	le(x,y)	Меньше или равно
$x\geq y$	ge(x,y)	Больше или равно

и логические операторы и функции:

Оператор	Функция	Описание
$x\&y$	and(x,y)	И (AND)
$x y$	or(x,y)	Логическое ИЛИ (OR)
$x\sim y$	not(x,y)	Логическое НЕ (NOT)

Например, $x > 0 \ \& \ x < 0$ – условие проверки на знак чисел x , y .

Кроме того, весьма полезны для решения задач, связанных с проверкой условий следующие арифметические функции:

Функция	Описание
<code>mod (x,y)</code>	Возвращает остаток от деления x на y без учета знака x
<code>rem (x,y)</code>	Возвращает остаток от деления x на y с учетом знака x
<code>ceil(x)</code>	Возвращает ближайшее большее или равное x целое число
<code>floor(x)</code>	Возвращает ближайшее меньшее или равное x целое число

Например, $x < 0 \ \& \ \text{mod}(x,2) == 0$ – условие проверки на четность отрицательных чисел.

```
>> mod(-5,2)
```

```
ans =
```

```
1
```

```
>> rem(-5,2)
```

```
ans =
```

```
-1
```

```
>> mod(4,2)
```

```
ans =
```

```
0
```

```
>> ceil(4.8)
```

```
ans =
```

```
5
```

```
>> floor(4.8)
```

```
ans =
```

```
4
```

Задача 1. Проверить введенное число на четность:

Для решения задачи необходимо:

- создать М файл функцию с одним выходным Y и одним входным x параметрами,
- сохранить ее в своей папке,
- для получения результата вызвать в командной строке М-функцию по имени (в примере Ch) относительно фактических параметров, например $Ch(6)$ и $Ch(5)$.

Программа решения задачи приведена на рис. 6.1.

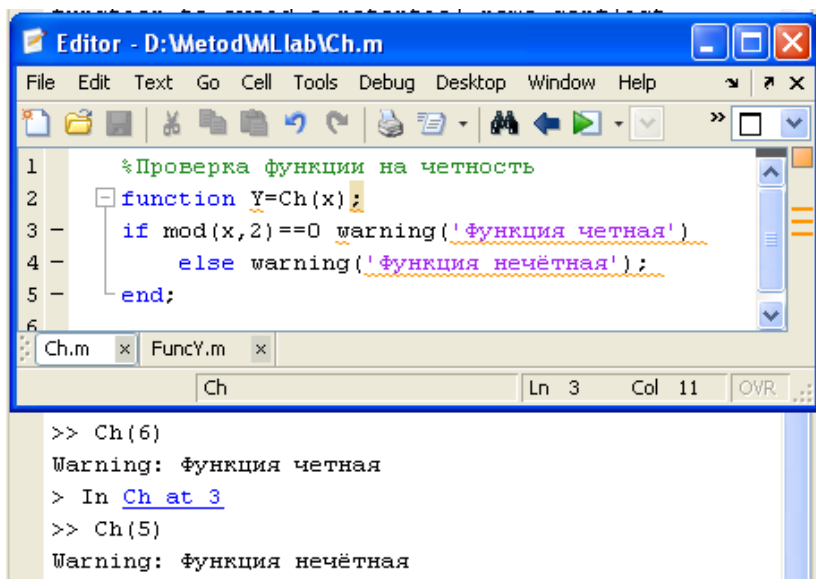


Рис. 6.1 Программа решения задачи 1.

Задача 2. Ввести 2 разных по значению числа. Если они оба положительные, то их значения заменить суммой и разностью, если оба отрицательные, то заменить их абсолютной величиной, если отрицательно только одно из них, то оба значения увеличить в 2 раза.

Поскольку условие задачи содержит два параметра, например a и b , и надо вывести два новых значения, то для решения задачи необходимо:

- создать М файл функцию, содержащую два выходных y_1 и y_2 и два входных a и b параметров, Выходные параметры задаются в виде вектора $[y_1, y_2]$, а входные a и b являются аргументами М-функции, например $D(a,b)$;
- сохранить файл $D.m$ в своей папке,
- для получения результатов вызвать в командной строке М-функцию по имени (в примере $D()$) относительно фактических параметров несколько раз, например $D(5,9)$, $D(-5,-9)$, $D(-5,9)$ для проверки работоспособности всех возможных вариантов задания.

Блок-схема и программа решения приведены на рис. 6.2.

Editor - C:\MATLAB701\work\D.m

```

1  function [y1 y2]=D(a,b)
2  - if a>0 & b>0 y1=a+b; y2=a-b;
3  - elseif a<0 & b<0 y1=abs(a); y2=abs(b);
4  - else y1=2*a; y2=2*b;
5  - end;
6  - [y1 y2]

```

Ln 6 Col 8 OVR

```

>> D(5,9)

ans =

    14    -4

```

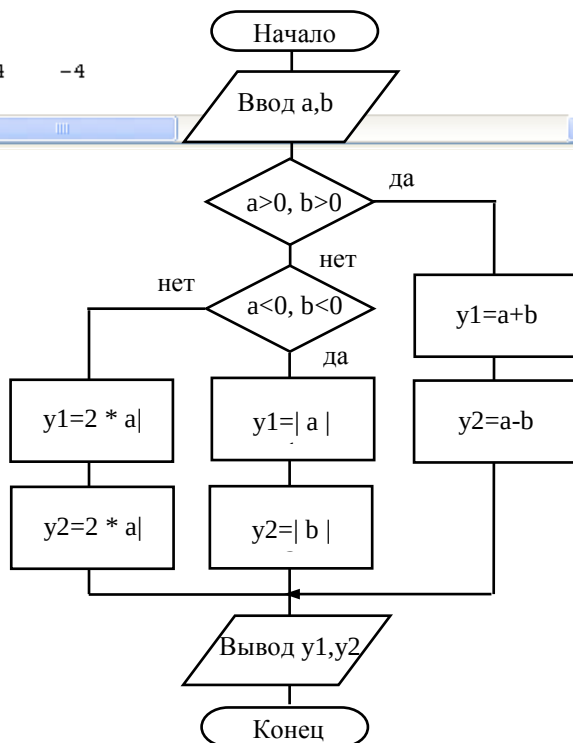


Рис. 6.2. Блок схема задачи:

Пример контрольного теста «Ветвления»

Установите понятие алгоритма линейной структуры:

Установите понятие алгоритма разветвленной структуры:

Установите соответствие:

if <условие> <оператор1> end;

if <условие> <оператор1>; else <оператор2>; end;

if <условие1> <оператор1>; elseif <условие2> <оператор2>;
else <оператор3>; end;

простейшая форма условного оператора

полная форма условного оператора

вложенная форма условного оператора

Укажите все операторы сравнения в MATLAB:

== <> >= ~= =

Результат операции mod(7,2)

7 1 3.5000 2 0

Укажите 3 правильных логических выражений языка MATLAB:

x<5 x=0 mod(x,2)==0 x>5 | x<0 x==1 & y>1

Условие $-1 \leq x < 3$ на языке MATLAB:

(x<3) or (x>=-1)

x<3 & x>=-1

(x>-1) and (x<=3)

x>-1 | x<3

(x>-1) and (x<=3)

Условие $x > 4$, $x \leq 0$ на языке MATLAB:

x<=0 | x>4

(x<0) and (x>4)

x>4 & x<0

(x>4) or (x<=0)

x<=0 | x>4

Укажите строку кода на языке MATLAB, соответствующую

$$\overbrace{y=x^2} \quad \overbrace{y=\ln(\sqrt{x})} \quad \overbrace{y=x^2}$$

задаче:

1

3

(переменная X - величина векторная)

if X>3 | X<1 Y=X^2 else Y=ln(sqrt(X)); end;

if X>1 & X<3 Y=log(sqrt(X)) else Y=X*X; end;

if X>3 | X<1 Y=X.^2 else Y=ln(sqrt(X)); end;

if X>1 & X<3 Y=log(sqrt(X)) else Y=X.^2;

if X>3 | X<1 Y=X.^2; else Y=log(sqrt(X)); end;

Строка кода на языке MATLAB, соответствующая формуле:

$$f = \begin{cases} 5\cos(x), & \text{при } x \leq 2 \\ a - \ln x^2, & \text{при } x > 2 \end{cases}$$

(переменная X - величина векторная)

if X>2 then F=a-ln(X*X); else F=5*sqr(cos(X));

if X>2 F=a-ln(X^2); else f:=5*cos^2(x); end;

if x>2 F=a-ln(X.^2); else F=5*cos(X)*cos(X); end;

if X>2 F=a-log(X.^2); else F=5*cos(X).^2; end;

if x<=2 then F=5*sqr(cos(X)); else F=a-ln(sqr(X)); end;

Фрагмент сценария позволяет в массиве целых чисел X1, X2,...Xn найти:

% ...

m=X(1);

for i=2:n

if X(i)>m m=X(i); end;

end;

наибольший элемент массива X

наименьший элемент массива X

номер максимального элемента массива X

номер минимального элемента массива X

старший элемент массива X

7. Циклические структуры

7.1. Подготовка к работе

По указанной литературе изучить:

- средства и особенности программирования MATLAB;
- циклические структуры и операторы их реализующие.

7.2. Задание и порядок выполнения работы

1. Запустить систему MATLAB и ввести текстовый комментарий "Управляющие структуры".
2. Задача 1. Создать М-файл для расчета суммы или произведения конечного ряда, заданного по варианту в табл. 7.1. При расчете использовать оператор цикла *for* с параметром. Результаты вычисления вывести в виде таблицы.

Таблица 7.1. Варианты заданий

N	Задание 1	N	Задание 1
	$a = 2.8; b = 0.7; x = 5.1$		$a = 2.8; b = 0.7; x = 5.1$
1	$z = \prod_{k=1}^{12} (\cos k + \sin k)$	7	$p = \prod_{n=1}^{15} \cos nx$
2	$z = \sum_{k=1}^{10} (\cos ak + \sin b)$	8	$y = \sum_{k=1}^{15} (\ln a + k \ln b)$
3	$y = \prod_{z=1}^{10} z^2 \cos(x + bz)$	9	$v = \prod_{n=1}^{12} \cos na^2 b^2$
4	$u = \sum_{n=1}^{12} n \sin^2 n^2 a$	10	$y = \sum_{k=1}^{15} \operatorname{tga}^2 k$
5	$z = \prod_{n=1}^{14} (\cos an + n)$	11	$t = \prod_{z=1}^{15} z^2 \cos(x + z)$
6	$t = \sum_{n=1}^{12} x \sin(n \cdot a)$	12	$u = \sum_{n=1}^{15} n \sin^2(n^2 a)$

3. Задача 2. Создать М-файл для расчета функции, заданной в таблице 7.2. Используя символьные вычисления задать функцию в виде ряда с точностью E. Определить члены ряда, их сумму и минимальное число членов ряда, обеспечивающих представление функции с заданной точностью. Результаты вычислений представить в виде таблицы.

Таблица 7.2. Варианты заданий

N	Задание 2		
	Функция	E	x
1	$\sin x = \sum_{n=0}^{\infty} (-1)^n x^{2n+1} / (2n+1)!$	0,005	1,2
2	$\sinh x = \sum_{n=0}^{\infty} x^{2n+1} / (2n+1)!$	0,002	0,9
3	$\cos x = \sum_{n=0}^{\infty} (-1)^n x^{2n} / (2n)!$	0,003	1,4
4	$\cosh x = \sum_{n=0}^{\infty} x^{2n} / (2n)!$	0,004	0,7
5	$\operatorname{atan} x = \sum_{n=0}^{\infty} (-1)^n x^{2n+1} / (2n+1)$	0,001	0,6
6	$\ln(x+1) = \sum_{n=0}^{\infty} (-1)^n x^{n+1} / (n+1)$	0,005	0,8
7	$x \sin x = \sum_{n=0}^{\infty} (-1)^n x^{2n+2} / (2n+1)!$	0,004	1,5
8	$x \cos x = \sum_{n=0}^{\infty} (-1)^n x^{2n+1} / (2n)!$	0,002	1,2
9	$x \sinh x = \sum_{n=0}^{\infty} x^{2n+2} / (2n+1)!$	0,003	0,9
10	$x \cosh x = \sum_{n=0}^{\infty} x^{2n+1} / (2n)!$	0,001	1,4

11	$e^x = \sum_{n=0}^{\infty} x^n / (n)!$	0,005	0,9
12	$x \ln(x + 1) = \sum_{n=0}^{\infty} (-1)^n x^{n+2} / (n + 1)$	0,004	0,7

4. Программы и блок-схемы поместить в отчет, созданный в текстовом редакторе MS Word. Сохранить его в своей папке.
5. Сделать выводы о разновидностях разветвляющих структур и форматах записи оператора, реализующего эти структуры в системе MATLAB.

7.3. Методические указания по выполнению задания

Процессы, в которых многократно повторяются отдельные участки вычислений, называются циклическими. Каждое повторение происходит при новых значениях переменных. Различают:

- регулярные циклы, в которых число повторений известно;
- итерационные циклы, в которых число повторений неизвестно и выход из цикла осуществляется при выполнении некоторого условия (обычно, достижении заданной точности вычисления). Для циклических вычислений используются операторы `for` и `while`.

Цикл типа `for...end` с заданным числом повторений

Формат оператора

`for` var=<выражение>

Инструкция_1, Инструкция_2;

`end`;

Цикл типа `while ...end` с неизвестным числом повторений

Формат оператора

`while`<выражение>

Инструкция_1, Инструкция_2;

`end`;

Задача 1. Рассмотрим пример вычисления суммы членов ряда:

$$y = \sum_{n=1}^{10} \sin(x \cdot n), \quad \text{при } x = 0.5$$

Функцию можно рассчитать как
 $y = \sin(x*1) + \sin(x*2) + \sin(x*3) + \sin(x*4) + \dots + \sin(x*10)$

$$y_0 = 0$$

$$y_1 = y_0 + \sin(x*1)$$

$$y_2 = y_1 + \sin(x*2)$$

$$y_3 = y_2 + \sin(x*3)$$

$$\dots\dots\dots$$

$$y_{10} = y_9 + \sin(x*10)$$

На рис. 7.1. приведена блок-схема, программа для решения задачи с помощью М-файла и результаты, подтверждающие решение задачи.

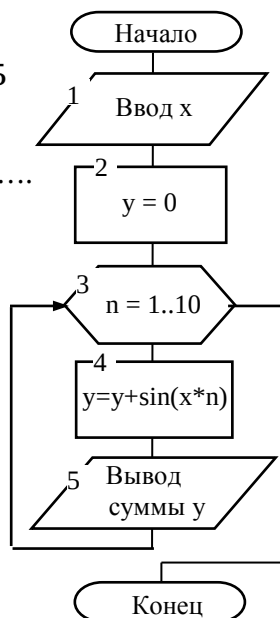


Рис. 7.1. Блок-схема и программа, реализующая задачу

```

>> N=10;    % Ввод исходных данных
>> Ysum;    % Вывод результата вычислений
Y =

```

Columns 1 through 6

0 0.4794 1.3209 2.3184 3.2277 3.8262

Columns 7 through 11

3.9673 3.6165 2.8597 1.8822 0.9232

В случае накопления произведения членов ряда, например

$$p = \prod_{n=1}^{10} \cos(x \cdot n), \quad \text{при } x = 0.5$$

Функцию можно рассчитать как $p = \cos(x*1) * \cos(x*2) * \cos(x*3) * \cos(x*4) * \dots * \cos(x*10)$

$$p_0 = 1$$

$$p_1 = p_0 * \cos(x*1)$$

$$p_2 = p_1 * \cos(x*2)$$

$$p_3 = p_2 * \cos(x*3)$$

.....

$$p_{10} = p_9 * \cos(x*10)$$

Блок-схема, реализующая вычислительный процесс, будет аналогична той, что представлена на рис. 7.3. Отличия в блоках 2 ($p=1$) и 4 (оператор “+” заменяется оператором “*”).

Задача 2. Рассмотрим расчет функции, представленной в табл. 7.2, разложив ее в ряд с заданной точностью E . Определить минимально необходимое число членов ряда, сами члены ряда и их сумму, обеспечивающие расчет функции с заданной точностью E .

$$\text{Представим функцию } e^{-x} = \sum_{n=0}^{\infty} (-1)^n x^n / (n)! \text{ в виде}$$

ряда с точностью 0,001 при $x=0,8$.

Как известно из математики функцию $y(x)$ можно разложить в ряд и рассчитать ее путем сложения первых нескольких n членом ряда.

$$\text{Например, } y = a_0 + a_1 + a_2 + a_3 + a_4 + \dots + a_n + \dots$$

Функция e^{-x} может быть разложена в ряд Тейлора. Ряд бесконечный, сходящийся, и рассчитывается путем сложения n членов ряда. Для организации циклических вычислений необходимо рассчитывать члены ряда, сравнивать их абсолютные значения с заданной точностью $|a_n| > E$ и при выполнении условия

сравнения производить их суммирование. Цикл завершается, когда n -ый член ряда оказывается меньше, чем заданная точность $|a_n| \leq E$.

Для написания программы сначала в символьном виде представим в командной строке n -й член ряда заданной функции:

```
>> syms x;      % символьное представление данных
>> n=0:8;      % ранжированная переменная n для первых 9 членов ряда
>> a=(-1).^n.*x.^n./factorial(n)
>> % n-й член ряда заданный формулой за знаком суммы
a =
[1, -x, x^2/2, -x^3/6, x^4/24, -x^5/120, x^6/720, -x^7/5040, x^8/40320]
a_0 a_1 a_2 a_3 a_4 a_5 a_6 a_7 a_8
```

Как видно, начальными значениями для расчета функции e^{-x} являются: $n=0$, $a_0=1$, $s_0=a_0$.

На рис.7.2. и 7.3 приведены блок-схема, программа решения задачи с помощью М-файла и результаты, подтверждающие решение задачи.

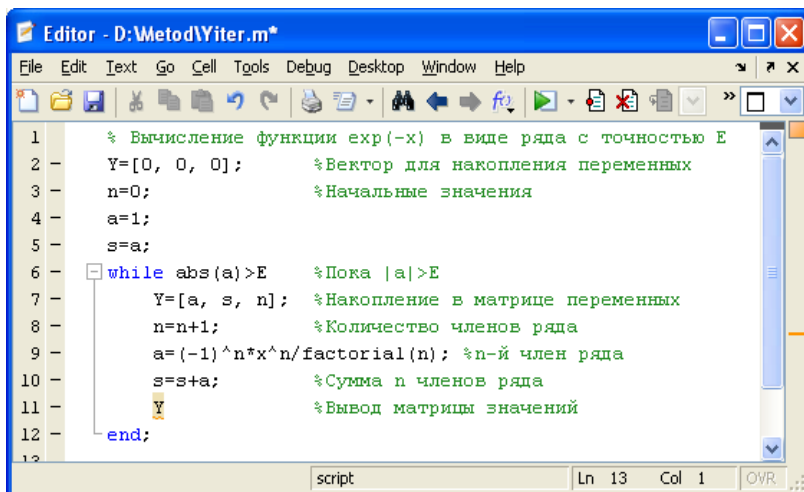


Рис. 7.2. Программа решения задачи

```

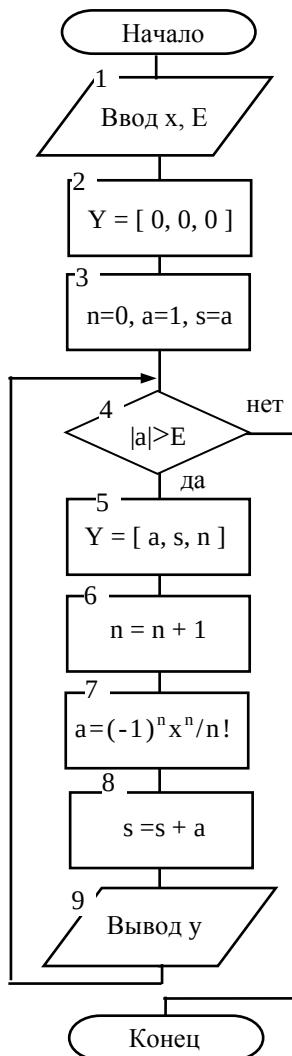
>> % исходные данные
>> x=0.8;
>> Yiter
Y =
    1    1    0
Y =
   -0.8000    0.2000    1.0000
Y =
    0.3200    0.5200    2.0000
Y =
   -0.0853    0.4347    3.0000
Y =
    0.0171    0.4517    4.0000
Y =
   -0.0027    0.4490    5.0000

>> % Проверка правильности решения
задачи
>> exp(-x)
ans =
    0.4493
>> 0.4493-0.4490
ans =
    3.0000e-004
>> % Погрешность вычисления =
0,0003 << 0,001

```

Как видно из полученного результата, что для представления функции e^{-x} с точностью вычисления $E=0.001$ при $x=0.8$ достаточно сложить первые 6 членов ряда ($a_0, a_1 \dots a_5$).

Измените переменную x и точность вычисления E . Проанализируйте результат представления заданной функции. Сделайте выводы, как погрешность вычисления влияет на результат представления функции.



7.3. Блок-схема Задачи

Пример контрольного теста «Циклы»

Установите понятие алгоритма циклической структуры:

Установите понятие регулярного цикла:

Установите понятие итеративного цикла:

Оператор цикла for ... служит для организации:

итерационных циклов с постусловием

итерационных циклов с предусловием

любых видов циклических процессов

циклов с заранее известным числом повторений

все ответы правильные

Оператор цикла while ... служит для организации:

циклов с заранее известным числом повторений

все ответы правильные

итерационных циклов с постусловием

итерационных циклов с предусловием

любых видов циклических процессов

Установите соответствие:

for i=1 : 10 <операторы>; end;

for i=10 : 1 <операторы>; end;

while <условие> <операторы>; end;

инкрементный вариант оператора цикла с параметром

декрементный вариант оператора цикла с параметром

оператор цикла с предусловием

Значение m в массиве A=[1 -2 3 1 2 -3 3 -1] в результате вы-

% ...

s=0; k=0;

for i=1:8

if A(i)>0 s=s+A(i); k=k+1; end;

end;

m=s/k;

полнения сценария равно:

10 2 5 0.5000 4

Значение k в массиве G=[1 -6 3 -1 2 1 3 -1] в результате вы-

полнения сценария равно:

```
% ...  
s=0; k=0;  
for i=1:8  
    if abs(G(i))==1 s=s+G(i); k=k+1; end;  
end;  
1    2    3    -6    4
```

Фрагмент сценария в массиве целых чисел F1, F2,...Fn позволяет найти ...

```
% ...  
s=0; n=1; k=0;  
while n<7  
    if F(n)<0 s=s+F(n); k=k+1; end;  
    n=n+1;  
end;
```

сумму чисел
сумму положительных чисел
все отрицательные числа
сумму отрицательных чисел
количество всех чисел

Фрагмент сценария в массиве целых чисел F1, F2,...Fn позволяет найти:

```
% ...  
s=0; n=1; k=0;  
while n<7  
    if F(n)<0 s=s+F(n); k=k+1; end;  
    n=n+1;  
end;
```

сумму чисел
сумму положительных чисел
все отрицательные числа
сумму отрицательных чисел
количество чисел

8. Моделирование электрических цепей

8.1. Подготовка к работе

По указанной литературе изучить:

- свойства объектов подбиблиотеки *Foundation Library* библиотеки *Simscape*:
Electrical Elements, Electrical Sensor, Electrical Sources.
- линейные цепи постоянного тока,
- линейные цепи синусоидального тока.

8.2. Задание и порядок выполнения работы

1. Задача 1. Создать модель электрической цепи постоянного тока, где величина источника напряжения U и сопротивление цепи R заданы по варианту в таблице 8.1. Измерить величину тока в электрической цепи.

Таблица 8.1. Варианты заданий

N	Напряже- ние (U), В	Сопротивле- ние (R), Ом	N	Напряже- ние (U), В	Сопротивле- ние (R), Ом
1	100	40	9	180	85
2	110	50	10	190	70
3	120	55	11	200	70
4	130	35	12	210	90
5	140	65	13	120	65
6	150	80	14	150	35
7	160	60	15	180	50
8	170	80	16	160	60

Для этого:

- рассчитать величину тока в цепи, используя закон Ома;
- создать модель электрической цепи постоянного тока с использованием блоков *Simscape*;
- сохранить модель и сравнить результат моделирования с вычисленным значением;
- скопировать в отчет модель и результаты проверки.

2. Задача 2. Создать модель электрической цепи переменного тока с активным сопротивлением. Варианты заданий приве-

дены в таблице 8.1, где напряжение для задачи 2 принимается в качестве амплитудного значения U_m . Вычислить комплексные значения тока и напряжения в электрической цепи с частотой $f=50$ Гц. Для этого:

- рассчитать величину тока в цепи, используя закон Ома;
- создать модель электрической цепи переменного тока с использованием блоков *SimElectronics*;
- построить временные диаграммы переменных величин;
- сохранить модель и сравнить результаты моделирования с вычисленным значением;
- скопировать в отчет модель, диаграммы и результаты проверки решения.

3. Сделать выводы о возможностях моделирования электрических цепей средствами пакета *SimElectronics*.

8.3. Методические указания по выполнению задания 1

1. Используя закон Ома для электрической цепи постоянного тока, вычислите величину тока цепи для заданного (по варианту из табл. 8.1) напряжения источника питания U и сопротивления цепи R . Для этого в командной строке MATLAB введите данные, например, $U=220$ В и $R=100$ Ом:

```
>>% Ввод исходных данных  
>> U=220;  
>> R=100;  
>> I=U/R % закон Ома  
I =
```

```
2.2000
```

2. С помощью пакета расширения *SimElectronics* создайте модель устройства (рис. 8.1), реализующего закон Ома для цепи постоянного тока. В таблице 8.2 указаны названия блоков, папки в которых находятся нужные блоки для моделирования и параметры блоков, которые нужно изменить:

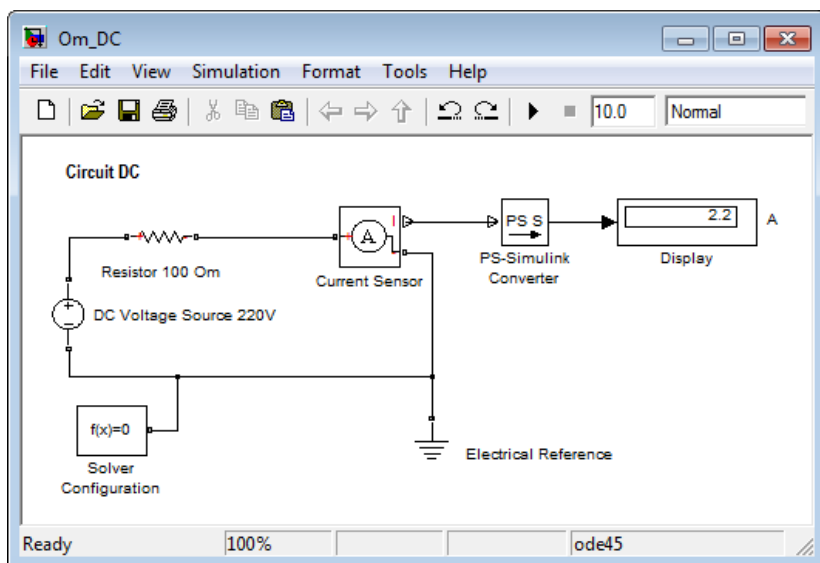


Рис. 8.1. Моделирование электрической цепи постоянного тока DC

Таблица 8.2. Значения параметров блоков моделирования

Название и назначение блоков	Размещение в браузере	Параметры блоков
Источник постоянного напряжения <i>DC Voltage Source</i>	<i>Simscape / Foundation Library / Electrical Sources</i>	<i>Constant voltage: 220V</i> //значение по варианту
Сопротивление <i>Resistor</i>	<i>Simscape / Foundation Library / Electrical Elements</i>	<i>Resistance: 100 Ohm</i> //значение по варианту
Блок решения <i>Solver Configuration</i>	<i>Simscape / Utilities</i>	По умолчанию
Амперметр <i>Current Sensor</i>	<i>Simscape / Foundation Library / Electrical Sensor</i>	По умолчанию
Преобразователь <i>PS-Simulink Converter</i>	<i>Simscape / Utilities</i>	По умолчанию
Регистратор <i>Display</i>	<i>Simulink / Sinks</i>	По умолчанию

Электрическое за- земление <i>Electrical Reference</i>	<i>Simscape / Foun- dation Library / Electrical Elements</i>	По умолчанию
---	--	--------------

3. Установите для каждого блока соответствующие свойства. Окна параметров блоков приведены на рис. 8.2 – 7.4.

Блок *DC Voltage Source* является идеальным источником напряжения, который поддерживает постоянное напряжение (DC) на выходных клеммах устройства, независимо от величины тока, протекающего через него. У идеального источника напряжения внутреннее сопротивление равно нулю. А выходное напряжение может принимать любое вещественное значение.

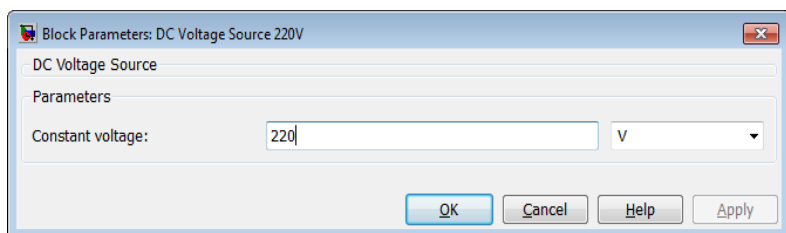


Рис. 8.2. Окно установки параметров блока *DC Voltage Source*

С помощью блока *Resistor* (рис. 8.3) устанавливается номинал сопротивления, которое связывает ток и напряжение соотношением $V = I * R$, где R – постоянное сопротивление, Ом. Положительные и отрицательные клеммы резистора обозначаются знаками + и - соответственно. По умолчанию, напряжение на резисторе задается $V (+) - V (-)$. Ток имеет положительное направление при прохождении через устройство от положительного полюса к отрицательному полюсу. Это гарантирует, что мощность, потребляемая резистором всегда положительна.

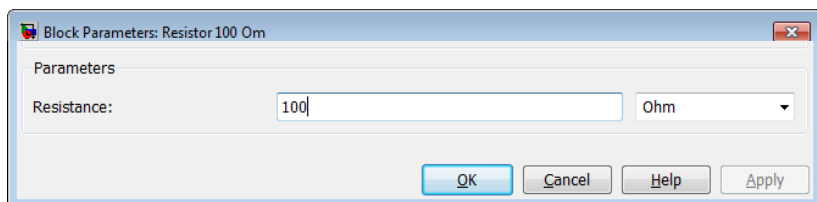


Рис. 8.3. Окно установки параметров блока *Resistor*

Каждая физическая цепь, в которую включаются блоки библиотеки *Simscape*, должна иметь один блок решения *Solver Configuration* – блок конфигурации решателя. Этот блок решает, какие параметры, подключенных в модель блоков, нужны для создания моделей с помощью блоков библиотеки *Simscape*. Блок *Solver Configuration* имеет один порт, с помощью которого он может быть подключен в любом месте к модели.

Начинается моделирование от устойчивого состояния *Start simulation from steady state*. При этом решатель с учетом начальных условий пытается найти стационарное состояние системы. По умолчанию время вычисления установлено $1 \cdot 10^{-9}$ с, что достаточно в большинстве случаев (рис. 8.4).

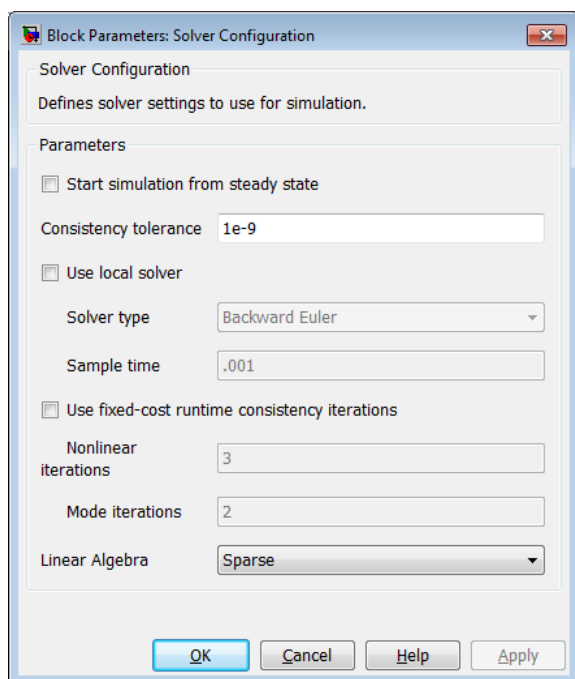


Рис. 8.4. Окно установки параметров блока *Solver Configuration*

Сенсор тока (амперметр) *Current Sensor* представляет собой идеальный датчик тока, который преобразует измеренный ток электрической цепи в физический сигнал, пропорциональ-

ный току. Соединения + и - это электрические порты, посредством которых датчик подключается к цепи. Амперметр, подключенный к порту, выводит результат измерения.

Блок конверсии *PS-Simulink Converter* преобразует физический сигнал в выходной сигнал Simulink. Он приводит форматы данных в соответствие с форматами различных пакетов расширения. Например, вход блока *Display (Simulink)* с выходом сенсора *Current Sensor (Simscape)*.

Блок *Electrical Reference* является электрическим опорным портом, служащим для подключения электрической земли.

3. Сохраните модель и запустите ее исполнение. Результаты вычислений отобразятся в дисплее (рис. 8.1). Скопируйте модель в отчет и сделайте выводы по проделанной работе.

8.4. Методические указания по выполнению задания 2

1. Используя закон Ома для электрической цепи переменного тока, вычислите величину переменного тока по заданным значениям амплитуды напряжения источника питания U_m и активного сопротивления цепи R . Для этого в командной строке *MATLAB* введите исходные данные, например,

$U_m=220$ В, $R=100$ Ом, $f=50$ Гц:

```
>> % Ввод исходных данных
>> f=50;
>> t=1;
>> Um=220;
>> R=100;
>> % угловая частота
>> w=2*pi*f;
>> % мгновенное значение
>> U=Um*cos(w*t)
U =
    220
>> % действующее значение U
>> Ud=Um/sqrt(2)
Ud =
    155.5635
```

```

>> % комплексное значение гармонического напряжения
>> U=Um*cos(w*t)+j*Um*sin(w*t)
U =
    2.2000e+002 +4.3217e-013i
>> % действительная часть a
>> a=Um*cos(w*t)
a =
    220
>> % мнимая часть b
>> b=Um*sin(w*t)
b =
    +4.3217e-013
>> % комплексное значение тока по закону Ома
>> I=U/R
I =
    2.2000 + 0.0000i
>> % комплексная гармоническая функция
>> u=Um*exp(j*(w*t))
u =
    2.2000e+002 +4.3217e-013i
>> % модуль (абсолютное значение) комплексного напряжения
>> abs(U)
ans =
    220
>> % угол комплексного напряжения
>> angle(U)
ans =
    1.9644e-015

```

2. В *Simulink* создайте модель устройства (рис. 8.5), реализующего закон Ома для цепи переменного тока с активным сопротивлением R . Для этого:

- создайте новое окно модели и скопируйте в него уже существующую модель DC ;
- замените источник питания постоянного тока на источник переменного тока AC .
- добавьте в модель недостающие элементы, приведенные на рис. 8.5 и в таблице 8.3.

В таблице 8.3 указаны параметры блоков моделирования:

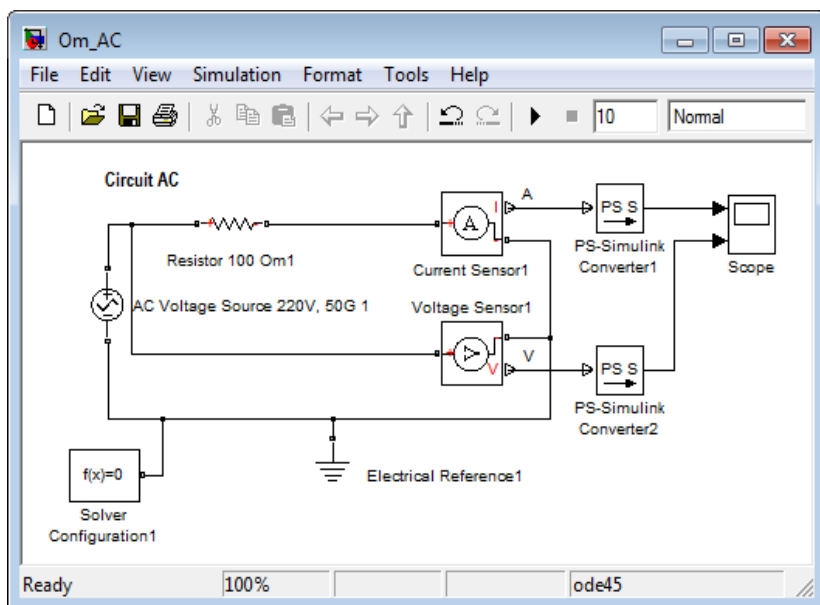


Рис. 8.5. Моделирование электрической цепи переменного тока AC

Таблица 8.3. Значения параметров блоков моделирования

Название и назначение блоков	Размещение в браузере	Параметры блоков
Источник переменного напряжения <i>AC Voltage Source</i>	<i>Simscape / Foundation Library / Electrical Sources</i>	<i>Peak amplitude: 220 V</i> //значение по варианту <i>Frequency: 50 Hz</i>
Сопrotивление <i>Resistor</i>	<i>Simscape / Foundation Library / Electrical Elements</i>	<i>Resistance: 100 Ohm</i> //значение по варианту
Блок решения <i>SolverConfiguration1</i>	<i>Simscape / Utilities</i>	По умолчанию
Амперметр <i>Current Sensor1</i>	<i>Simscape / Foundation Library / Electrical Sensor</i>	По умолчанию
Вольтметр <i>Voltage Sensor1</i>	<i>Simscape / Foundation Library / Electrical Sensor</i>	По умолчанию

Преобразователь <i>PS-Simulink Converter1,2</i>	<i>Simscape / Utilities</i>	По умолчанию
Электрическое за- земление <i>Electrical Reference1</i>	<i>Simscape / Foundation Library / Electrical Elements</i>	По умолчанию
Регистратор Scope	<i>Simulink / Sinks</i>	Number of axes: 2

3. Установите для каждого блока соответствующие свойства. Окна параметров блоков приведены на рис. 8.6 и 8.7.

Блок *AC Voltage Source* (рис. 8.6) является идеальным источником напряжения переменного тока, который обеспечивает синусоидальное напряжение на его выходных клеммах, независимое от величины тока, протекающего через него. Выходное напряжение определяется по формуле

$$U = U_m \cdot \sin(\omega \cdot t + \varphi),$$

где U_m является амплитудным значением, $\omega = 2 \cdot \pi f$ является угловой частотой в рад / с, и φ – фазовым сдвигом в радианах. Установите амплитудное значение напряжения по варианту из табл. 8.1, частоту (*frequency*) равной 50 Гц, а сдвиг фазы примем 0.

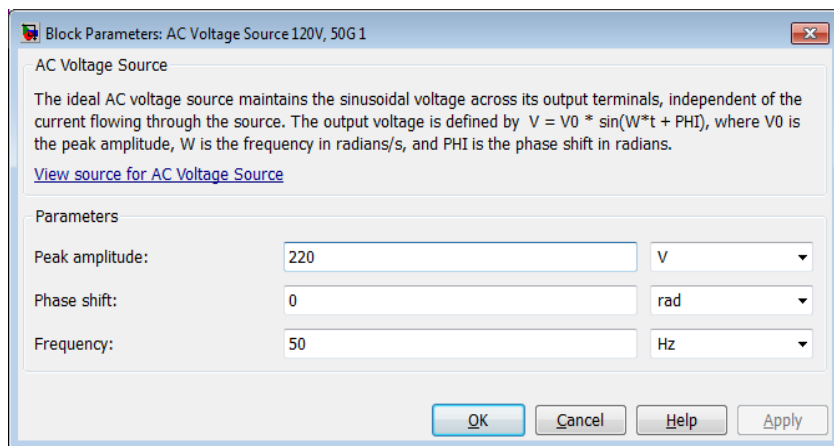


Рис. 8.6. Окно установки параметров блока *AC Voltage Source*
Сенсор напряжения (вольтметр) *Voltage Sensor* представ-

ляет собой идеальный датчик напряжения, который преобразует измеренное напряжение между любыми точками электрической цепи в физический сигнал, пропорциональный напряжению. Соединения + и - это электрические порты, посредством которых датчик подключается к цепи и выводит значение напряжения.

Виртуальный осциллограф *Scope* позволяет представить результаты моделирования в виде временных диаграмм с оцифрованной масштабной сеткой. Осциллограф имеет свою панель инструментов (рис. 8.7), которая содержит следующие кнопки:

- печать (1), параметры (2),
- масштаб по осям (3),
- автоматическое масштабирование по осям (4),
- сохранение текущих установок (5),
- режим «плавающего» осциллографа (6).

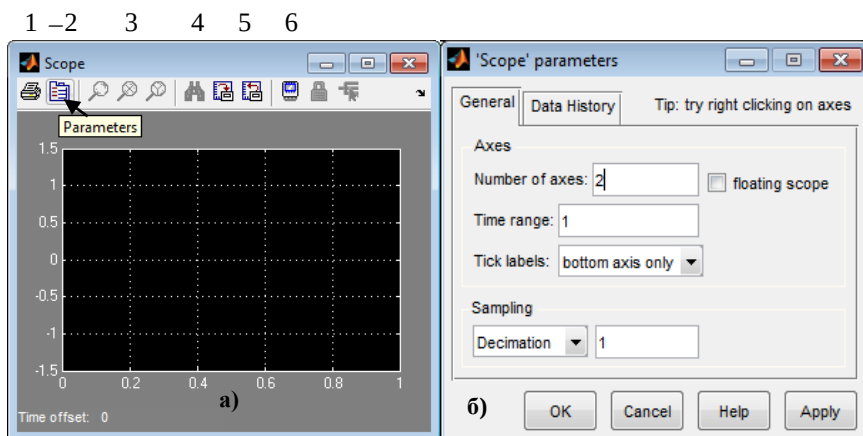


Рис. 8.7. Работа с виртуальным осциллографом

Для отображения двух осциллограмм в одном окне:

- открыть двойным щелчком по блоку *Scope* его демонстрационное окно (рис. 8.7 а);
- кнопкой *Parameters* (Параметры) окна *Scope* вызвать окно свойств (рис. 8.7 б), в котором на главной вкладке *General* установить число осей *Number of axes* осциллографа 2;
- установить временной диапазон (*Time range*) отображения временных диаграмм =1, щелкнуть по кнопке *OK* или *Apply*.

Конверторы сигналов *PS-Simulink Converter* позволяют соединять *Simulink*-осциллографы с выходами *Simcsape*-сенсоров тока и напряжения (амперметра и вольтметра).

4. Сохраните модель и запустите ее исполнение. Чтобы посмотреть осциллограммы работы *Scope*, достаточно двойным щелчком открыть окно установки параметров осциллографа и результаты моделирования отобразятся в окне (рис. 8.8).

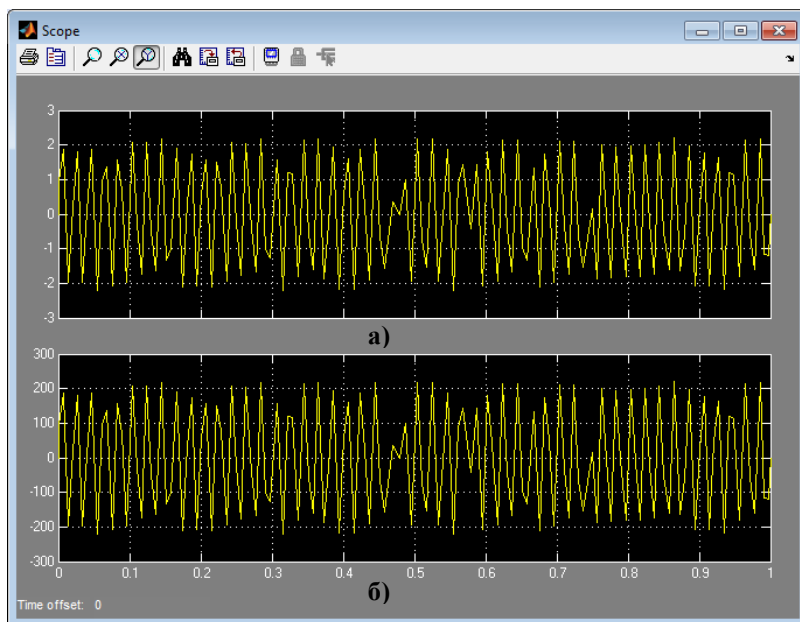


Рис. 8.8. Осциллограммы выходных синусоидальных сигналов:
а) тока и б) напряжения

5. Скопируйте модель и осциллограммы в отчет.

Как видно по осциллограммам ток и напряжение носят гармонический характер. Амплитудные значения тока достигают $\pm 2,2$ А, а напряжения – ± 220 В, что хорошо согласуется с законом Ома. Следует заметить, что данный источник формирует сигнал, по форме несколько отличающийся от синусоидального.

6. Создайте модель (рис. 8.9), вычисляющую **комплексные значения** напряжения и переменного тока электрической цепи с активным сопротивлением R .

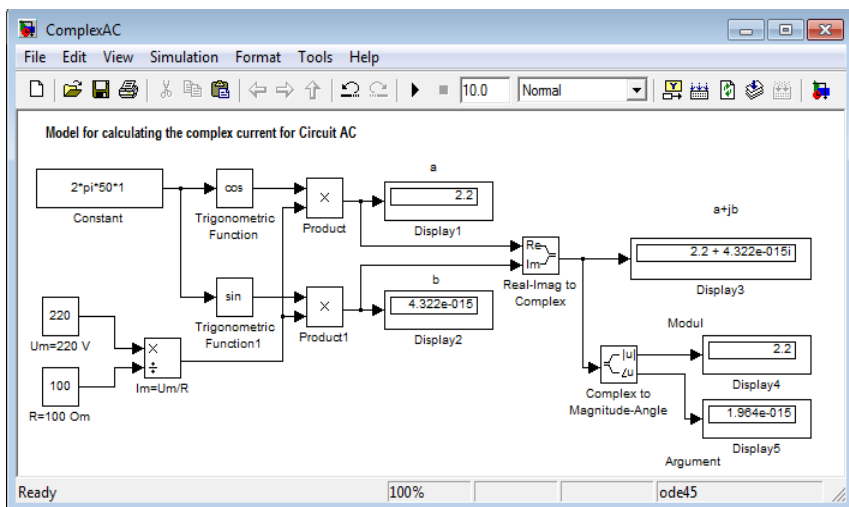


Рис. 8.9. Модель, вычисляющая комплексный ток и напряжение

В таблице 8.4 указаны параметры блоков моделирования.

Таблица 8.4. Значения параметров блоков моделирования

Название блоков	Размещение в браузере	Параметры блоков
<i>Constant, Constant1, Constant2</i>	<i>Simulink / Sources</i>	<i>Constant=2*pi*50*1 Constant1=220 //по варианту Constant2=100 //по варианту</i>
Блок деления <i>Divide</i>	<i>Simulink / Math Operations</i>	<i>Number of inputs: */</i>
Блок умножения <i>Product ,1</i>	<i>Simulink / Math Operations</i>	<i>Number of inputs: 2</i>
Блок <i>Trigonometric Function ,1</i>	<i>Simulink / Math Operations</i>	<i>Function cos Function1 sin</i>
Блок <i>Real-Imag to Complex</i>	<i>Simulink / Math Operations</i>	По умолчанию
Блок <i>Complex to</i>	<i>Simulink / Math</i>	По умолчанию

<i>Magnitude-Angle</i>	<i>Operations</i>	
Регистраторы <i>Display1-5</i>	<i>Simulink / Sinks</i>	По умолчанию

Как известно [7], комплексное значение напряжения вычисляется по формуле:

$$U = U_m \cdot \cos(\omega \cdot t) + j \cdot U_m \cdot \sin(\omega \cdot t),$$

где $\omega = 2 \cdot \pi f$ – угловая частота,

$a = U_m \cdot \cos(\omega t)$ – действительная часть,

$b = U_m \cdot \sin(\omega t)$ – мнимая часть.

```
>> Umod=sqrt(a^2+b^2) % модуль
```

```
Umod =
```

```
220
```

```
>> tanFi=b/a % фаза
```

```
tanFi
```

```
1.9644e-015
```

В подбиблиотеке *Math Operations* библиотеки *Simulink* имеются блоки для работы с комплексными числами. Название блоков и их назначение приведены в табл. 8.5:

Таблица 8.5. Значения параметров *Complex* блоков

Название блоков	Назначение
<i>Real-Image to Complex</i>	вычисляет комплексное число по его действительной и мнимой части
<i>Complex to Magnitude-Angle</i>	вычисляет модуль и аргумент комплексного числа

7. Сохраните модель и запустите ее исполнение. Результаты вычислений отобразятся на дисплеях *Display1-5* (рис. 8.9).

8. Сопоставьте полученные результаты с вычисленными значениями в режиме прямых вычислений.

9. Скопируйте модель и результаты проверки ее работоспособности в отчет и сделайте выводы по проделанной работе.

Как показала проверка моделирования электрических цепей постоянного и переменного токов с помощью встроенных средств *MATLAB*, модели собраны и функционируют верно. Все они работоспособны и выполняют поставленные задачи.

Пример контрольного теста «Электрические цепи»

Укажите закона Ома для электрической цепи постоянного тока:

$$U=I+R$$

$$U=I \cdot R$$

$$U=I/R$$

$$U=I \cdot G$$

$$I=U \cdot R$$

Укажите действующее значение:

$$U=U_m/R$$

$$U=U_m/2$$

$$U=U_m/\sqrt{2}$$

$$U=U_m \cdot \sin(t)$$

$$I_m=U_m \cdot R$$

Укажите комплексную переменную:

$$A=A_m \cdot \sin(w \cdot t) + i \cdot \cos(w \cdot t)$$

$$A=A_m \cdot \sin^2(w \cdot t) + i \cdot \cos^2(w \cdot t)$$

$$A=A_m \cdot \cos(w \cdot t) + i \cdot A_m \cdot \sin(w \cdot t)$$

$$A=A_m \cdot \cos^2(w \cdot t) + i \cdot A_m \cdot \sin_2(w \cdot t)$$

$$A=A_m \cdot \exp(-i \cdot (w \cdot t))$$

Укажите функцию *MATLAB*, вычисляющую модуль комплексного числа:

`mod(U)`

`abs(U)`

`module(U)`

`|U|`

`sqrt(U)`

Укажите функцию *MATLAB*, вычисляющую фазу комплексного числа:

`arg(U)`

`argument(U)`

`angle(U)`

`fi(U)`

`arg(a,b)`